

Computational Physics

Learnable activation functions in physics-informed neural networks for solving partial differential equations

Afrah Farea^{ID,*}, Mustafa Serdar Celebi^{ID}

Department of Computational Science and Engineering, Istanbul Technical University, Istanbul 34469, Turkey

ARTICLE INFO

The review of this paper was arranged by Prof. Andrew Hazel

Keywords:

Partial differential equations
Physics informed neural networks
Multilayer perceptrons
Kolmogorov-Arnold networks
Learnable activation function

ABSTRACT

Physics-Informed Neural Networks (PINNs) have emerged as a promising approach for solving Partial Differential Equations (PDEs). However, they face challenges related to spectral bias (the tendency to learn low-frequency components while struggling with high-frequency features) and unstable convergence dynamics (mainly stemming from the multi-objective nature of the PINN loss function). These limitations impact their accuracy for solving problems involving rapid oscillations, sharp gradients, and complex boundary behaviors. We systematically investigate learnable activation functions as a solution to these challenges, comparing Multilayer Perceptrons (MLPs) using fixed and learnable activation functions against Kolmogorov-Arnold Networks (KANs) that employ learnable basis functions. Our evaluation spans diverse PDE types, including linear and non-linear wave problems, mixed-physics systems, and fluid dynamics. Using empirical Neural Tangent Kernel (NTK) analysis and Hessian eigenvalue decomposition, we assess spectral bias and convergence stability of the models. Our results reveal a trade-off between expressivity and training convergence stability. While learnable activation functions work well in simpler architectures, they encounter scalability issues in complex networks due to the higher functional dimensionality. Counterintuitively, we find that low spectral bias alone does not guarantee better accuracy, as functions with broader NTK eigenvalue spectra may exhibit convergence instability. We demonstrate that activation function selection remains inherently problem-specific, with different bases showing distinct advantages for particular PDE characteristics. We believe these insights will help in the design of more robust neural PDE solvers.

1. Introduction

Solving partial differential equations (PDEs) underpins numerous scientific and engineering applications, from fluid dynamics to electromagnetic wave propagation. While traditional numerical methods like finite element analysis provide accurate solutions, they face computational bottlenecks in high-dimensional spaces and require extensive mesh generation. Physics-Informed Neural Networks (PINNs) [47] offer a mesh-free alternative that directly incorporates physical laws into the learning process. By embedding governing equations and boundary conditions as part of the loss function, PINNs offer a physics- and data-efficient approach for both forward and inverse PDEs, enabling the discovery of unknown physical parameters even with limited datasets [13,28,26,32,42,4,45,76].

Achieving accurate solutions and maintaining convergence stability are ongoing challenges in PINNs. A significant issue is spectral bias [46,72,10], which leads to slow convergence especially in wave

propagation (e.g., Helmholtz, Wave, Klein-Gordon), multi-physics flows (e.g., Convection-diffusion), and in fluid dynamics (e.g., Cavity flow). Researchers have explored alternative architectures and different activation functions, such as those mentioned in [35,44,20,39,49,62], among others. More specifically, the Kolmogorov Arnold Network (KAN) [39] offers flexibility through different basis functions (e.g., splines or polynomials) and trainable scaling factors. Recent studies suggest that KANs exhibit reduced spectral bias, improving their ability to capture high-frequency components [67,30]. Despite these advances, the spectral bias problem remains a challenge for MLPs and KANs [11,70,22,27].

Handling PDEs with different frequencies presents a challenge for traditional numerical methods as well. A solution to this issue is the Multigrid Method (MG) [9,61], which uses hierarchical grids to manage various frequency components. This method employs iterative solvers to initially smooth high-frequency components on finer grids while correcting low-frequency errors on coarser grids. In the context of neural network architectures, such as KANs, the incorporation of learnable ba-

* Corresponding author.

E-mail addresses: farea16@itu.edu.tr (A. Farea), mscelebi@itu.edu.tr (M.S. Celebi).

sing functions introduces adaptive grid sizes. This leads to an important question: Can learnable activation functions mitigate spectral bias and enhance convergence stability in PINNs when approximating multi-scale PDEs? Does low spectral bias indeed guarantee improved accuracy and stability, or do other factors dominate the optimization landscape? Understanding these relationships is crucial for developing robust neural PDE solvers that can effectively handle the multi-scale nature of complex physical systems.

In this paper, our main contributions are:

1. **Exploration of learnable activations:** We extensively compare KANs and MLPs with both fixed and learnable activation functions (in this paper, we use the terms “learnable” and “trainable” interchangeably), revealing their respective strengths in training convergence and testing error.
2. **Study of convergence and spectral bias behavior:** Using the empirical Neural Tangent Kernel (NTK) and maximum eigenvalue of the loss Hessian, we analyze each model’s capacity to approximate high-frequency patterns and convergence stability. Additionally, we investigate whether low spectral bias guarantees good testing outcomes.
3. **Application to diverse PDEs:** We evaluate performance across PDEs with varying spectral properties: oscillatory problems (e.g., Helmholtz [41,38,16,64] and Wave [6,52]), nonlinear wave (e.g., Klein-Gordon [25,69]), mixed-physics systems (e.g., Convection-diffusion [50,5]), and complex fluid dynamics (e.g., Cavity [55,23,59,73]) problems, using various activation and basis functions (e.g., Tanh, parameterized-Tanh, B-splines, GRBFs, Fourier, Chebyshev, and Jacobi polynomials)

The source code and pre-trained models used in this study are publicly available on GitHub at https://github.com/afrah/pinn_learnable_activation.git and Zenodo [17], facilitating reproducibility and further exploration.

2. Related work

Adaptive activation functions (e.g., learnable or trainable activation function) play a crucial role in neural networks by enhancing their expressivity and reducing the need for manual hyperparameter tuning. However, this adaptability comes at the cost of increased training complexity, optimization difficulty, and computational overhead.

Notably, Apicella et al. [3] classified learnable activation functions into two categories. The first category is parameterized standard activation functions, in which specific parameters are learned during training. For example, the original Swish activation proposed in [48] has the form $x \cdot \sigma(\beta x)$ where, β is a constant or learnable parameter. The second category consists of functions based on ensemble methods, where multiple candidate activation functions or models are combined to create more flexible and adaptable activation function $f(x)$ such that $f(x) = \sum_{i=1}^h \alpha_i g_i(x)$ where, $g_i(x)$ is a linear mapping, and α_i are learnable weights. Sütfield et al. [57] proposed Adaptive Blending Units (ABUs) that combine a set of basic activation functions through learnable blending weights, allowing the neural network to learn an optimal activation shape for each layer during training. Wang et al. [63] followed a similar approach using g_i as a standard activation function and experimentally showed that this method outperforms the fixed activation function. However, the success of this method depends on the diversity and suitability of the candidate activation functions, and the complexity increases with an increasing number of candidate functions.

Jagtap et al. [25] demonstrated that using adaptive activation functions significantly enhances the training process. Liu et al. [39] demonstrated that KAN models with B-spline basis functions can accurately solve the Poisson equation, outperforming MLPs in accuracy and parameter efficiency. Zang et al. [75] showed that KAN performs better on regular functions, but MLP generally outperforms KAN on functions with

irregularities like discontinuities and singularities. Hong et al. [22] developed a piecewise linear spline activation function, known as the Hat function (inspired by finite element methods), to balance the learning behavior across frequency spectra. Recent works, such as [67,54,51,68], showed that KANs can improve convergence and reduce spectral bias compared to MLPs, thereby enhancing their ability to represent complex functions.

Despite these advances, significant limitations persist in current research, as they are primarily focused on straightforward PDEs or small-scale issues. Additionally, there is a lack of systematic comparisons among various types of PDEs with different spectral characteristics in the literature.

3. Problem formulation and neural network methods

Our methodology includes: (a) Study of PINNs with architectures such as MLPs and KANs. This section will detail these architectures. (b) Comprehensive evaluation of these architectures with various activation functions across PDEs (in Section 4) that have varying spectral characteristics. The results of this testing will be presented in Section 5 and 6. (c) Lastly, we provide a theoretical analysis of the results using NTK and Hessian eigenvalue decomposition to understand convergence behavior in Section 6.

3.1. Differential equation

The general differential equation has the form:

$$D[u(\mathbf{x}); \alpha] = f(\mathbf{x}), \quad \mathbf{x} \in \Omega, \quad (1)$$

$$B_k[u(\mathbf{x})] = g_k(\mathbf{x}), \quad \mathbf{x} \in \Gamma_k \subset \partial\Omega, \quad k = 1, 2, \dots, n_b$$

where, D is the differential operator, B_k is a set of boundary operators, $f(\mathbf{x})$ is the forcing function, $g_k(\mathbf{x})$ is a set of boundary functions, α is a set of parameters used in the differential equation and $u(\mathbf{x})$ is the solution of the differential equation given input $\mathbf{x}$ in the solution domain Ω with boundary $\partial\Omega$. This work considers five PDEs as use cases discussed in Section 4.

3.2. Multilayer perceptron representation

We start with standard MLPs as our baseline model. Fig. 1 (left) shows the structure of an MLP model with learnable weights and activation functions (fixed or learnable). The effectiveness of MLPs is theoretically grounded in the universal approximation theorem [14], which asserts that a feedforward network with a single hidden layer containing $k > N(\epsilon)$ neurons can approximate any continuous function on a compact subset of $\mathbb{R}^N$ to within a specified error ϵ . In its simplest form, an MLP consists of a finite set of inputs x_j , each associated with a learnable weight θ_{ij} , representing the connection strength between input j and neuron i , along with a bias term that adjusts the neuron’s activation threshold. The output of an MLP can be represented as:

$$f(\mathbf{x}) = \sum_L w_{i_L}^{(L)} \sigma_{i_L}^{(L-1)} + b^{(L)} \quad (2)$$

where, each $\sigma_i^{(\ell)}$ is recursively defined as:

$$\sigma_i^{(\ell)} = \begin{cases} x_i, & \ell = 0 \\ \phi^{(\ell)} \left(\sum_j w_{ij}^{(\ell)} \sigma_j^{(\ell-1)} + b_i^{(\ell)} \right), & \ell = 1, \dots, L-1 \end{cases}$$

where, L is the number of layers and $\phi(\cdot)$ denotes the activation function, w_{ij} and b_i are the weights and biases respectively. The activation function must be a simple, nonlinear, and differentiable function to enable the network to capture complex patterns and compute gradients efficiently during backpropagation. Ideally, the activation function is smooth enough to ensure stable gradient behavior during optimization

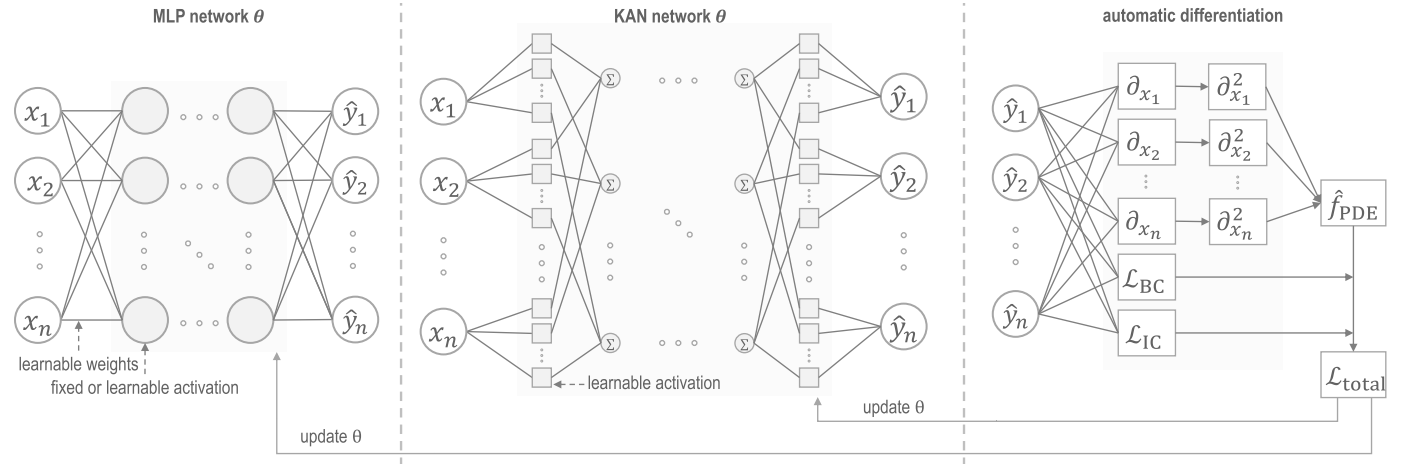


Fig. 1. This diagram presents a high-level comparison between the MLP and KAN architectures within the PINN framework. The MLP network uses learnable weights with nonlinear activation functions during training. In contrast, KAN incorporates learnable activation functions. The MLP and KAN networks pass the outputs to the Physics module, which uses automatic differentiation (AD) to calculate physics loss.

Algorithm 1 A KAN model with a general basis function.

Require: x : input data of shape $[\text{batch_size}, l_0]$, network: $[l_0, l_1, \dots, l_n]$, d : degree of basis function
Ensure: y : output of the model for input x

- 1: Initialize an empty list layers
- 2: **for** each layer (l_i, l_{i+1}) in network **do**
- 3: Create a new KAN layer L_i with input dimension l_i , output dimension l_{i+1}
- 4: Initialize learnable coefficients coeffs_{ij} for each layer L_i
- 5: Append layer L_i to layers
- 6: **end for**
- 7: **Forward Pass:**
- 8: If required, normalize x to the domain of the basis function
- 9: **for** each layer L_i in layers **do**
- 10: Compute $B(x) \leftarrow \text{basis_function}(x, d)$
- 11: Compute $y \leftarrow \sum_{j=0}^d \text{coeffs}_{ij} \cdot B_j(x)$
- 12: **end for** **return** y
- 13: **Function:** $\text{basis_function}(x, d)$
- 14: Generate basis function $B(x)$ from input x and degree d
- 15: **return** $B(x)$

and appropriately bounded to mitigate gradients while maintaining non-saturating derivatives to avoid vanishing gradients. For PDE solving, the activation function must satisfy additional constraints beyond typical machine learning applications: it requires well-defined higher-order derivatives for accurate physics loss computation and sufficient smoothness to ensure stable gradient propagation through the PDE operators.

3.3. Kolmogorov-Arnold approximation

Fig. 1 (middle) illustrates the structure of KAN with learnable activation functions. According to Kolmogorov-Arnold representation theorem [31], any multivariate continuous function f of n variables on a bounded domain can be represented by a superposition of a finite number of continuous functions with a single variable and the summation operation. More formally, for any continuous function $f: \mathbb{R}^n \rightarrow \mathbb{R}$, there exist continuous functions ϕ_j and ψ such that:

$$f(\mathbf{x}) = \sum_{i=1}^{2n+1} \psi_i \left(\sum_{j=1}^n \phi_{ij}(x_j) \right) \quad (3)$$

where, $\mathbf{x}$ is the input variable, $\psi_i: \mathbb{R} \rightarrow \mathbb{R}$ and $\phi_{ij}: [0, 1] \rightarrow \mathbb{R}$ are parameterized as continuous learnable univariate functions. Liu et al. [39] extend this formula to multiple layers such that:

$$\mathbf{x}^{(0)} = \mathbf{x}$$

$$x_i^{(\ell)} = \begin{cases} \sum_{j=1}^{n_{\ell-1}} \phi_{ij}^{(\ell)}(x_j^{(\ell-1)}), & \ell = 1, \dots, L-1 \\ \sum_{j=1}^{n_{L-1}} \psi_{ij}(x_j^{(L-1)}), & \ell = L \end{cases}$$

The network structure in KANs learns a combination of simpler functions akin to the ensemble methods discussed in Section 2. Specifically, the base learners ($g_i(x)$) and the weighted sum α_i in the ensemble methods correspond to the inner univariate functions ϕ_{ij} and the outer functions ψ_{ij} in KANs, respectively. The key difference between KAN and ensemble functions, such as Adaptive Blending Units (ABUs) [57], lies in their approach to function representation. While KAN focuses on building a universal approximation by decomposing a multivariate function into a sum of univariate functions using the Kolmogorov-Arnold representation theorem, the ensemble methods blend multiple activation functions, optimizing their combinations through learned weights to adaptively change the activation shape during training. However, the Kolmogorov-Arnold representation theorem guarantees representation for any continuous function but provides no guarantees about learnability in finite-parameter, gradient-based settings.

In their original work, Liu et al. [39] used the SiLU activation function to capture the non-linear relationships that could not be captured by the B-spline network as follows:

$$\phi(x) = \lambda_0 \text{silu}(x) + \lambda_1 \text{spline}(x) = \lambda_0 \text{silu}(x) + \lambda_1 \sum_{i=1}^{d+k-1} c_i B_i^d(x) \quad (4)$$

where, d is the spline order, k is the grid size, λ_0 , λ_1 and c_i are trainable parameters, and $B_i^d(x)$ is the B-spline function.

Although B-splines [15] are commonly used due to their flexibility and smoothness properties, alternative functional forms such as Chebyshev polynomials [56], Radial Basis Functions (RBF) [34,58], Fourier [71], wavelet [8,53], etc. can also be used for the inner function ϕ . These alternatives may offer specialized basis functions based on the nature and application of the problem, making KAN a general framework for function approximation. More generally, Algorithm 1 shows the steps for parametrization of the activation function within the KAN formulation. The main idea is to map the inputs of each neural network layer to a higher-dimensional space using a suitable basis function, multiply the result by learnable parameters, and optimize iteratively.

In this study, we examined several well-known basis functions, as shown in Table 1, for solving PDEs. The selection criteria are based on their unique characteristics and mathematical properties. For instance, while cubic or high-degree B-splines are differentiable and smooth, the Fourier basis functions have their differentiability tied directly to their coefficients, making them versatile for approximating periodic functions.

Table 1

KAN basis functions considered in the study and Tanh activation function where, O : the number of parameters in terms of asymptotic notation, n : the number of units per hidden layer, L : the number of hidden layers, k : the grid size, and d : the degree of the polynomial basis function.

Basis Function	Equation	Local Support	Global Support	Node Requirement	Numerical stability	Smoothness	Boundedness	Number of Parameters	Suitability
B-spline	$B_{i,n}(x) = \frac{x-l_i}{l_{i+n}-l_i} B_{i,n-1}(x) + \frac{l_{i+n+1}-x}{l_{i+n+1}-l_{i+1}} B_{i+1,n-1}(x)$	✓	✗	✓	✓	✓	Locally compact support	$\mathcal{O}(n^2 \cdot L \cdot (k + d - 1))$	Piecewise Polynomial Approximation, Curve Fitting
Radial Basis Function	$e^{-\frac{\ x-c\ ^2}{\sigma^2}}$	✓	✗	✓	✓	✓	$(0, 1]$	$\mathcal{O}(n^2 \cdot L + k)$	Interpolation, Spatial Modeling
Fourier	$\sum_k a_k \cos(kx) + b_k \sin(kx)$	✗	✓	✗	✓	✓	$[-1, 1]$ per basis	$\mathcal{O}(n^2 \cdot L \cdot k)$	Periodic Functions, Signal Processing
Chebyshev	$T_d(x) = 2xT_{d-1}(x) + T_{d-2}(x)$	✗	✓	✓	✓	✓	$[-1, 1]$	$\mathcal{O}(n^2 \cdot L \cdot (d + 1))$	Spectral Methods, Polynomial Approximation
Jacobi	$P_d^{(\alpha,\beta)}(x) = \frac{(-1)^d}{2^d d!} (1-x)^{-\alpha} (1+x)^{-\beta} D^d [(1-x)^{\alpha+d} (1+x)^{\beta+d}]$	✗	✓	✓	✓	✓	$[-1, 1]$	$\mathcal{O}(n^2 \cdot L \cdot (d + 1))$	Weighted Polynomial Approximation
Tanh	$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$	✗	✓	✗	✓	✓	$[-1, 1]$	$\mathcal{O}(n^2 \cdot L)$	Smooth, global pattern

On the other hand, learnable activations can also introduce challenges, particularly in training stability. The increased complexity may lead to issues such as vanishing or exploding gradients during backpropagation, especially in deep networks or when higher-degree polynomials are used. Furthermore, numerical stability may become a concern, as many of these functions are sensitive to inputs outside their well-defined intervals, which can potentially lead to significant calculation errors.

The Tanh activation function is well-known for suffering from vanishing and exploding gradients when inputs deviate significantly from zero, as its gradient diminishes toward zero when $\tanh(x) \rightarrow \pm 1$. Similarly, polynomial basis functions (such as Chebyshev and Jacobi polynomials) can lead to gradient issues. Near-zero polynomial values often cause vanishing gradients, while higher-degree polynomials may cause exploding gradients during backpropagation. This sensitivity to input scale and polynomial degree can destabilize the training process.

Radial Basis Functions (RBFs) are also susceptible to vanishing gradients when the input is far from the center c_i , especially if the scaling factor σ is not optimal, as the gradient approaches zero in these regions. While higher-degree splines generally offer stability, improper scaling or poor placement of knots can induce both vanishing and exploding gradients. Additionally, Fourier basis functions can cause gradient explosions when high frequencies dominate, resulting in rapid oscillations. This oscillatory behavior significantly increases the gradient magnitude, which can make training unstable.

These challenges highlight the importance of carefully selecting and scaling hyperparameters and activation functions to maintain stable gradients and prevent issues during backpropagation.

3.4. Physics informed neural networks

We integrate PINNs with both MLPs and KANs to directly incorporate physical laws into the network's learning process by embedding governing equations into the loss function. Fig. 1 (right) shows the structure of a PINN, where the outputs of the network (either MLP or KAN) are used to solve PDEs by enforcing physics-based constraints through automatic differentiation. The idea is to recover the unique solution of the PDE by penalizing the boundary and initial conditions as data-driven supervised learning and choosing finitely many representative points from the interior that satisfy the differential equation. For instance, to approximate the solution of the PDE in Eq. (1), we train the PINN model by minimizing the loss function of the form:

$$\begin{aligned} \mathcal{L}(\theta) &= \operatorname{argmin}_{\theta} \sum_{k=1}^n \lambda_k \mathcal{L}_k(\theta) \\ &= \operatorname{argmin}_{\theta} \left(\lambda_1 \mathcal{L}_1(D[u(\mathbf{x}); \alpha] - f(\mathbf{x})) + \sum_{k=2}^n \lambda_k \mathcal{L}_k(B_k[u(\mathbf{x})] - g_k(\mathbf{x})) \right) \end{aligned} \quad (5)$$

where, n is the number of loss terms, and λ_k is the weighting coefficient of the respective loss term.

No prior knowledge of the actual solution $u(\mathbf{x})$ is required, as minimizing Eq. (5) yields the desired solution. Eq. (5) reduces the multi-objective constraints of the PDE Eq. (1) into a single constraint. An important observation is that this loss function includes terms with different physical scales, particularly when high-order derivatives are present in the physics loss term. This variation can significantly impact the quality and stability of the training process. Therefore, various adaptive methods have been proposed in the literature to balance the loss weighting terms λ_k based on different criteria, such as [21,40,65,66,7,2,77,78], to name a few.

4. Use cases

This section outlines the selected 1D and 2D PDE equations, their boundary conditions, and the loss functions used for evaluation. These equations encompass various problems, such as the Helmholtz and Wave oscillatory problems, the Klein-Gordon equation representing nonlinear waves, Convection-diffusion in mixed-physics systems, and Cavity flow involving complex fluid dynamics.

4.1. 1D wave equation

The wave equation is a second-order hyperbolic PDE that models the propagation of waves, such as sound or vibrations, through a medium. We consider the time-dependent 1D wave equation of the form:

$$\begin{aligned} u_{tt}(t, x) - c^2 u_{xx}(t, x) &= 0 & (t, x) \in \Omega \\ u(t, x_0) &= f_1(t, x) & (t, x) \text{ on } \Gamma_0 \\ u(t, x_1) &= f_2(t, x) & (t, x) \text{ on } \Gamma_1 \\ u(0, x) &= g(t, x) & (t, x) \in \Omega \\ u_t(0, x) &= h(t, x) & (t, x) \in \partial\Omega \end{aligned} \quad (6)$$

where, c is the wave speed. We set $c = 2$, $a = 0.5$, $f_1 = f_2 = 0$ and exact solution of the form, $u(t, x) = \sin(\pi x) \cos(c\pi t) + 0.5 \sin(2c\pi x) \cos(4c\pi t)$ on the domain $(t, x) \in [0, 1] \times [0, 1]$. The equation simplifies to:

$$\begin{aligned} u_{tt}(t, x) - 4u_{xx}(t, x) &= 0 & (t, x) \in \Omega = [0, 1] \times [0, 1] \\ u(t, 0) &= u(t, 1) = 0 \\ u(0, x) &= \sin(\pi x) + 0.5 \sin(4\pi x) & x \in [0, 1] \\ u_t(0, x) &= 0 \end{aligned} \quad (7)$$

The loss function for solving the problem with PINN is:

$$\begin{aligned}\mathcal{L}(\theta) &= \min_{\theta} (\lambda_1 \|\mathcal{L}_{phy}(\theta)\|_{\Omega} + \lambda_2 \|\mathcal{L}_{bc}(\theta)\|_{\Gamma_1} + \lambda_3 \|\mathcal{L}_{ic}(\theta)\|_{\Gamma_0}) \\ &= \min_{\theta} (\lambda_1 \mathcal{L} \|u_{\theta_{tt}}(t, x) - 4u_{\theta_{xx}}(t, x)\|_{\Omega} + \lambda_2 \mathcal{L} \|u(t, 0) + u(t, 1)\|_{\Gamma_1} \\ &\quad + \lambda_3 \mathcal{L} \|u(0, x)\|_{\Gamma_0} + \lambda_3 \mathcal{L} \|u_{\theta_t}(0, x)\|_{\partial\Omega})\end{aligned}\quad (8)$$

We set $\lambda_1 = 1.0$, $\lambda_2 = 100.0$, and $\lambda_3 = 100.0$. The difficulty of this problem lies in the presence of both low- and high-frequency components in the solution, as well as the nonlinearity introduced by the interaction between time and space.

4.2. Helmholtz equation

The Helmholtz equation is a time-independent form of the wave equation arising in many physical problems like vibrating membranes, acoustics, and electromagnetism. We consider a 2D Helmholtz equation of the form:

$$\begin{aligned}\Delta u(x, y) + k^2 u(x, y) &= q(x, y) \quad (x, y) \in \Omega \\ u(x, y) &= h(x, y) \quad (x, y) \in \Gamma_0\end{aligned}\quad (9)$$

where, Δ is the Laplacian operator ($\Delta u = u_{xx} + u_{yy}$), $q(x, y)$ is the forcing term, and k is the wavenumber. For simplicity, we assume Dirichlet boundary conditions ($h(x, y)$) equal to an exact solution of the form, $u(x, y) = \sin(a_1 \pi x) \sin(a_2 \pi y)$ corresponding to the source term $q(x, y) = u(x, y)[k^2 - (a_1 \pi)^2 - (a_2 \pi)^2]$ with $k = 1$, $a_1 = 1$, and $a_2 = 4$. The final form of the equation with the boundary and initial conditions are:

$$\begin{aligned}\Delta u(x, y) + u(x, y)[(a_1 \pi)^2 + (a_2 \pi)^2] &= 0 \quad (x, y) \in \Omega = [-1, 1] \times [-1, 1] \\ u(x, y) &= 0 \quad (x, y) \in \Gamma_0\end{aligned}\quad (10)$$

The loss function for solving the problem with PINN is:

$$\begin{aligned}\mathcal{L}(\theta) &= \min_{\theta} (\lambda_1 \|\mathcal{L}_{phy}(\theta)\|_{\Omega} \\ &\quad + \lambda_2 \|\mathcal{L}_{bc}(\theta)\|_{\Gamma_0}) \\ &= \min_{\theta} (\lambda_1 \mathcal{L} \|u_{\theta_{xx}}(x, y) + u_{\theta_{yy}}(x, y) + \alpha u_{\theta}(x, y)\|_{\Omega} \\ &\quad + \lambda_2 \mathcal{L} \|u_{\theta}(x, y) - u(x, y)\|_{\Gamma_0})\end{aligned}\quad (11)$$

We set $\lambda_1 = 1.0$, and $\lambda_2 = 10.0$.

4.3. Klein-Gordon equation

The Klein-Gordon equation is a second-order hyperbolic PDE that appears in various fields such as quantum field theory, general relativity, and nonlinear optics. We consider 1D nonlinear Klein-Gordon equation of the form:

$$\begin{aligned}u_{tt} - \alpha u_{xx} + \beta u + \gamma u^k &= 0 \quad (t, x) \in \Omega \\ u(t, x) &= g_1(t, x) \quad (t, x) \\ u_t(t, x) &= g_2(t, x) \quad (t, x) \\ u(0, x) &= h(t, x) \quad (t, x) \in \partial\Omega \times [0, T]\end{aligned}\quad (12)$$

where, α , β , γ and k are known constants. We set 1, 0, 1, 3 for α , β , γ and k respectively. We use the exact solution of the form, $u(t, x) = x \cos(5\pi t) + (tx)^3$. The final form of the equation with the boundary and initial conditions is:

$$\begin{aligned}u_{tt}(t, x) - \alpha u_{xx}(t, x) + \beta u(t, x) + u^3(t, x) &= 0 \quad (t, x) \in \Omega = [0, 1] \times [0, 1] \\ u(t, 0) &= 0 \\ u(t, 1) &= \cos(5\pi t) + (t)^3 \\ u(0, x) &= x \\ u_t(0, x) &= 0\end{aligned}\quad (13)$$

The loss function for solving the problem with PINN is:

$$\begin{aligned}\mathcal{L}(\theta) &= \min_{\theta} (\lambda_1 \|\mathcal{L}_{phy}(\theta)\|_{\Omega} \\ &\quad + \lambda_2 \|\mathcal{L}_{bc}(\theta)\|_{\Gamma_1} + \lambda_3 \|\mathcal{L}_{ic}(\theta)\|_{\Gamma_0}) \\ &= \min_{\theta} (\lambda_1 \mathcal{L} \|u_{\theta_{tt}}(t, x) - u_{\theta_{xx}}(t, x) + u^3(t, x)\|_{\Omega} \\ &\quad + \lambda_2 \mathcal{L} \|u(t, 0) + u(t, 1) + u(0, x)\|_{\Gamma_1} + \lambda_3 \mathcal{L} \|u_{\theta_t}(0, x)\|_{\partial\Omega})\end{aligned}\quad (14)$$

We set $\lambda_1 = 1.0$, $\lambda_2 = 50.0$, and $\lambda_3 = 50.0$.

4.4. Convection-diffusion equation

The convection-diffusion equation is a second-order PDE that models the transport of a substance, such as heat, in a medium, taking into account both convection (movement due to flow) and diffusion (spreading out due to concentration gradients). We consider the viscous 2D convection-diffusion equation of the form:

$$\begin{aligned}u_t + c_1 u_x + c_2 u_y - D \Delta u(x, y) &= 0 \quad (t, x, y) \in \Omega \\ u(t, \mathbf{x}) &= g_0(t, \mathbf{x}) \quad (t, \mathbf{x}) \in \Gamma_0 \\ u(t, \mathbf{x}) &= g_1(t, \mathbf{x}) \quad (t, \mathbf{x}) \in \Gamma_1 \\ u(0, \mathbf{x}) &= h(0, \mathbf{x}) \quad \mathbf{x} \in \partial\Omega \times [0, T]\end{aligned}\quad (15)$$

where, c_1 and c_2 are the convection velocities in the x and y directions respectively, D is the diffusion coefficient, and Δ is the Laplacian operator. We use an exact solution of the form $u(t, x, y) = \exp(-100((x - 0.5)^2 + (y - 0.5)^2)) \exp(-t)$ which describes a Gaussian pulse centered at $(x, y) = (0.5, 0.5)$ that decays over time. The corresponding to the initial condition is $h(0, x, y) = \exp(-100((x - 0.5)^2 + (y - 0.5)^2))$. For this particular problem, later timesteps are important because the system has evolved, and the solution becomes more diffuse and less concentrated.

We set $c_1 = 1.0$, $c_2 = 1.0$ (high convection), and $D = 0.01$ (low diffusion). The final form of the equation with the boundary and initial conditions is:

$$\begin{aligned}u_t(t, x, y) + c_1 u_x(t, x, y) + c_2 u_y(t, x, y) \\ - D (u_{xx}(t, x, y) + u_{yy}(t, x, y)) &= 0 \\ (t, x, y) \in \Omega &= [0, 1] \times [0, 1] \times [0, 1]\end{aligned}\quad (16)$$

$$\begin{aligned}u(t, x, y) &= g(t, x, y) \quad (t, x, y) \in \Omega \\ u(0, x, y) &= h(x, y) \quad (x, y) \in \partial\Omega \times [0, T]\end{aligned}$$

The loss function for solving the problem with PINN is:

$$\begin{aligned}\mathcal{L}(\theta) &= \min_{\theta} (\lambda_1 \|\mathcal{L}_{phy}(\theta)\|_{\Omega} + \lambda_2 \|\mathcal{L}_{bc}(\theta)\|_{\Gamma_1} + \lambda_3 \|\mathcal{L}_{ic}(\theta)\|_{\Gamma_0}) \\ &= \min_{\theta} (\lambda_1 \mathcal{L} \|u_{\theta_t}(t, x, y) + c_1 u_{\theta_x}(t, x, y) + c_2 u_{\theta_y}(t, x, y) \\ &\quad - D(u_{\theta_{xx}}(t, x, y) + u_{\theta_{yy}}(t, x, y))\|_{\Omega} \\ &\quad + \lambda_2 \mathcal{L} \|u(t, x, y) - g_1(t, x, y)\|_{\Gamma_1} + \lambda_3 \mathcal{L} \|u_{\theta}(0, x, y) - h(x, y)\|_{\Omega_0})\end{aligned}\quad (17)$$

We set $\lambda_1 = 1.0$, $\lambda_2 = 10.0$, and $\lambda_3 = 10.0$.

4.5. Time-dependent 2D cavity problem

This problem is a widely recognized benchmark in Computational Fluid Dynamics (CFD). It involves simulating the movement of cavity walls to drive fluid flow within the cavity. In this case study, the upper boundary moves at a constant velocity, creating a shear-driven flow inside the cavity, while the other boundaries remain stationary. The problem is governed by the unsteady, incompressible Navier-Stokes equations as follows:

$$\begin{aligned}\rho \left(\frac{\partial \mathbf{u}}{\partial t} + \mathbf{u} \cdot \nabla \mathbf{u} \right) &= -\nabla p + \mu \nabla^2 \mathbf{u} \quad \text{in } \Omega \in [0, 1] \\ \nabla \cdot \mathbf{u} &= 0 \quad \text{in } \Omega\end{aligned}\quad (18)$$

Table 2

KAN learnable activation function with the hyperparameters used in the study where a , b , a_{ijr} , b_{ijr} , and c_{ijr} are learnable parameters.

Type	Hyperparameters	Activation Function $f(z)$
Tanh	-	$f(z) = \tanh\left(\sum_{j=1}^{n_i} w_{ij} z_j + b_i\right)$
parametric-Tanh	-	$f(z) = \tanh\left(a \cdot \left(\sum_{j=1}^{n_i} w_{ij} z_j + b_i\right) + b\right)$
B-spline	$g = 8,$ $d = 3$	$f(z) = \sum_{i=1}^{n_i} \left(\sum_{j=1}^{n_{i-1}} \sum_{r=1}^{g+d+1} c_{ijr} \cdot B_r^3(z_j) \right)$ where: $B_r^3(z) = \begin{cases} \frac{1}{6}(z - t_r)^3 & \text{if } z \in [t_r, t_{r+1}) \\ \frac{1}{6}((t_{r+1} - z)^3 - 4(z - t_{r+1})^3) & \text{if } z \in [t_{r+1}, t_{r+2}) \\ \frac{1}{6}((t_{r+3} - z)^3 - 4(t_{r+2} - z)^3) & \text{if } z \in [t_{r+2}, t_{r+3}) \\ \frac{1}{6}(t_{r+4} - z)^3 & \text{if } z \in [t_{r+3}, t_{r+4}) \\ 0 & \text{otherwise} \end{cases}$
GRBF	$\sigma = 1,$ $k = 8$	$f(z) = \sum_{i=1}^{n_i} \left(\sum_{j=1}^{n_{i-1}} \sum_{r=1}^k c_{ijr} \cdot e^{-\frac{\ z_j - t_r\ ^2}{\sigma^2}} \right)$
Fourier	$k = 4$	$f(z) = \sum_{i=1}^{n_i} \left(\sum_{j=1}^{n_{i-1}} \sum_{r=1}^k [a_{ijr} \cos(kz_j) + b_{ijr} \sin(kz_j)] \right)$
Chebyshev	$d = 4$	$f(z) = \sum_{i=1}^{n_i} \left(\sum_{j=1}^{n_{i-1}} \sum_{r=0}^d c_{ijr} \cdot T_r(z_j) \right)$ where: $T_0(z) = 1, \quad T_1(z) = z,$ $T_2(z) = 2z^2 - 1,$ $T_3(z) = 4z^3 - 3z,$ $T_4(z) = 8z^4 - 8z^2 + 1$
Jacobi	$d = 4,$ $\alpha = 1,$ $\beta = 1$	$f(z) = \sum_{i=1}^{n_i} \left(\sum_{j=1}^{n_{i-1}} \sum_{r=0}^d c_{ijr} \cdot P_r^{(1,1)}(z_j) \right)$ where: $P_0^{(1,1)}(z) = 1, \quad P_1^{(1,1)}(z) = 2z,$ $P_2^{(1,1)}(z) = \frac{3}{2}(5z^2 - 1),$ $P_3^{(1,1)}(z) = \frac{5}{2}(7z^3 - 3z),$ $P_4^{(1,1)}(z) = \frac{35}{8}(9z^4 - 10z^2 + 1)$

$$\mathbf{u}(0, x) = 0 \quad \text{in } \Omega$$

$$\mathbf{u}(t, x_0) = 0 \quad \text{on } \Gamma_1$$

$$\mathbf{u}(t, x_l) = 1 \quad \text{on } \Gamma_0$$

The computation domain Ω is a two-dimensional square cavity $\Omega = (0, 1) \times (0, 1)$. Uniform discretization is used with grid $(N_x, N_y) = (100, 100)$ and 10 seconds total simulation time with 0.01s time interval. Γ_1 is the top Dirichlet boundary condition of the cavity with velocity tangent to this side, Γ_0 denotes the other three stationary sides, $\rho = 1056$, and $\mu = \frac{1}{Re} = 0.01$; where, Re is Reynolds number of the flow and ρ is fluid density. We used Ansys Fluent software with the finite volume method to generate the reference solution and compare the results with the neural network approximation. The loss function for solving the Cavity problem is as follows:

$$\mathcal{L}(\theta) = \lambda_1 \|\mathcal{L}_{phy}\|_{\Omega} + \lambda_2 (\|\mathcal{L}_{up}\|_{\Gamma_1} + \|\mathcal{L}_{bc_1}\|_{\Gamma_0}) + \lambda_3 \|\mathcal{L}_{u0}\|_{\Omega} \quad (19)$$

where, $\mathcal{L}_{phy}(\theta) = \mathcal{L}_{r_u} + \mathcal{L}_{r_v} + \mathcal{L}_{r_c}$. We select $\lambda_1 = 0.1$, $\lambda_2 = 2.0$, and $\lambda_3 = 4.0$ such that:

$$\mathcal{L}_{r_u}(\theta) = \text{MSE} \left[(\hat{u}_t + \hat{u} \hat{u}_x + \hat{v} \hat{u}_y) + \frac{1.0}{\rho} \hat{p}_x - \mu(\hat{u}_{xx} + \hat{u}_{yy}) \right]$$

$$\mathcal{L}_{r_v}(\theta) = \text{MSE} \left[(\hat{v}_t + (\hat{u} \hat{v}_x + \hat{v} \hat{v}_y) + \frac{1.0}{\rho} \hat{p}_y - \mu(\hat{v}_{xx} + \hat{v}_{yy}) \right]$$

$$\mathcal{L}_{r_c}(\theta) = \text{MSE} [\hat{u}_x + \hat{v}_y]$$

with the boundary and initial conditions:

$$\mathcal{L}_{up} = \text{MSE} [(1.0 - \hat{u} + (\hat{v}))]$$

$$\mathcal{L}_{bc_1} = \mathcal{L}_{\text{bottom, right, left}} = \text{MSE} [(\hat{u} + \hat{v})]$$

$$\mathcal{L}_{u0} = \text{MSE} [\hat{u} + (\hat{v}) + (\hat{p})]$$

5. Computational experiments

5.1. Experiment setup

Table 2 shows the hyperparameters for the KAN basis functions, which are explained in Table 1. Table 3 presents two groups of neural network architectures (Fig. 1) along with their corresponding settings used in this study, as well as the total number of learnable parameters for each model. We designate these architectures as A1 and A2, where A1 is designed with fewer parameters while A2 includes more parameters, thereby increasing model complexity and enhancing expressivity for function approximation.

The settings used in this study are selected to achieve optimal performance for moderately complex problems while minimizing the risk of overfitting and reducing computational complexity. Therefore, we employed identical training settings for shared parameters to ensure a fair comparison of the models. All models were trained using the PyTorch Adam optimizer with hyperparameters $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$, a constant learning rate of $\eta = 0.001$, and a weight decay of $10e^{-6}$. We applied Xavier initialization to initialize the weights of all models and minibatch training with a batch size of 128, sampled randomly for 60,000 iterations in all PDE cases.

In the MLP models, we tested two variants with different activation functions: standard Tanh and parametric-Tanh. The standard model used Tanh as its activation function, while the latter used $\text{Tanh}(ax + b)$, where a and b are learnable parameters initialized to 1 and 0, respectively. We evaluate the performance of the models using the relative L_2 -norm (Eq. (20)) and measure the time per iteration in seconds.

$$\text{Relative } L_2 \text{ norm} = \frac{\|y - \hat{y}\|_2}{\|y\|_2} * 100\% \quad (20)$$

For simplicity, throughout the remainder of the paper, we refer to the models by their activation or basis function along with the respective architecture identifier. For example, Tanh(A1) represents an MLP with Tanh activation function using A1 architecture, whereas B-spline(A2)

Table 3

Two different neural network architectures (A1 and A2) and their corresponding parameters for MLP and KAN models. Architectures A1 and A2 differ in the number of units (neurons) per layer and the overall complexity of the network, with A1 being smaller and A2 being larger, making A2 more expressive. Cells marked with “–” indicate that the particular configurations are not used.

Arch.	Case	Network	MLP		KAN				
			Tanh	Param-Tanh	B-spline	GRBF	Fourier	Chebyshev	Jacobi
A1	Helmholtz	[2, 30, 30, 30, 1]	1981	1983	18900	15120	15211	9450	9450
	Klein-Gordon	[2, 30, 30, 30, 1]	1981	1983	18900	15120	15211	9450	9450
	Wave	[2, 10, 30, 10, 1]	681	683	6300	5040	5091	3150	3150
	Convection-Diffusion	[3, 50, 50, 50, 1]	5351	5353	52000	41600	41751	26000	26000
	Cavity	[3, 50, 50, 50, 3]	5453	5455	53000	42400	42553	26500	26500
A2	Helmholtz	[2, 300, 300, 300, 1]	181801	181803	-	-	-	-	-
	Klein-Gordon	[2, 400, 400, 400, 1]	322401	322403	-	-	-	-	-
	Wave	[2, 300, 300, 300, 300, 1]	272101	272103	-	-	-	-	-
	Convection-Diffusion	[3, 300, 300, 300, 1]	182101	182103	-	-	-	-	-
	Cavity	[3, 300, 300, 300, 3]	182703	182705	-	-	-	-	-
	Helmholtz	[2, 150, 150, 150, 1]	-	-	454500	363600	364051	227250	227250
	Klein-Gordon	[2, 200, 200, 200, 1]	-	-	806000	644800	645401	403000	403000
	Wave	[2, 150, 150, 150, 150, 1]	-	-	679500	543600	544201	339750	339750
	Convection-Diffusion	[3, 150, 150, 150, 1]	-	-	456000	364800	365251	228000	228000
	Cavity	[3, 100, 100, 100, 3]	-	-	206000	164800	165103	103000	103000

Table 4

Testing relative L_2 error (in %) for the cases considered in our study. We used two neural network architecture sets (A1 and A2), detailed in Table 3. The **lowest** and the **second-lowest** errors are highlighted. For simplicity, we select the best model based on the accuracy of the first feature in each row, which corresponds to the velocity u . Cells marked with “–” indicate that the error exceeded 90%. t/it : time per iteration in seconds. (For interpretation of the colors in the table(s), the reader is referred to the web version of this article.)

Case		MLP				KAN									
		Tanh		param-Tanh		B-spline		GRBF		Fourier		Chebyshev		Jacobi	
		A1	A2	A1	A2	A1	A2	A1	A2	A1	A2	A1	A2	A1	A2
Helmholtz	u	6.13	6.62	5.51	4.52	1.93	3.03	17.9	8.11	3.09	–	5.17	–	19.5	–
	f	3.09	1.63	2.50	0.96	0.92	0.71	4.52	3.76	1.18	–	1.37	–	2.01	–
	t/it	0.0111	0.0116	0.0145	0.0153	0.0904	0.0907	0.0238	0.0243	0.0309	–	0.0934	–	0.1376	–
Klein-Gordon	u	1.58	9.48	11.4	18.6	5.23	13.2	15.0	25.8	9.25	–	12.7	–	8.36	–
	f	5.08	4.85	2.22	1.77	7.89	0.33	0.22	0.43	0.62	–	1.07	–	0.46	–
	t/it	0.0153	0.0131	0.0187	0.0154	0.1028	0.0841	0.0260	0.0254	0.0367	–	0.1044	–	0.1354	–
Wave	u	–	13.6	–	–	22.4	27.7	17.2	14.7	38.4	–	37.5	44.2	37.5	–
	t/it	–	0.0146	–	–	0.0823	0.1030	0.0217	0.0266	0.0294	–	0.0889	0.1167	0.1328	–
Convection Diffusion	u	12.6	13.6	12.1	11.6	11.7	12.3	11.5	12.8	15	–	17.5	–	29.9	–
	f	2.45	3.86	1.67	1.07	8.34	1.01	2.07	3.01	7.10	–	2.46	–	3.02	–
	t/it	0.0132	0.0170	0.0177	0.0174	0.0975	0.0991	0.0277	0.0299	0.0340	–	0.1005	–	0.1351	–
Cavity	u	13.9	10.8	13.5	15.7	7.87	4.09	13.1	13.5	25.2	40.9	17.2	14.6	16.9	37.1
	v	24.3	22.0	32.8	21.9	18.0	8.72	25.6	17.7	38.9	75.2	32.5	24.2	44.2	65.2
	p	25.9	24.4	45.8	33.5	25.4	21.3	28.2	29.7	52.2	–	37.1	52.2	54.1	–
	t/it	0.0282	0.0271	0.0333	0.0358	0.2030	0.1982	0.0537	0.0538	0.0771	0.0760	0.2137	0.2255	0.2923	0.2876

represents a KAN model with B-spline basis function using architecture A2. Although our results could be further optimized through more extensive hyperparameter tuning or an exhaustive architecture search, our focus in this work is on evaluating the models under consistent settings.

Finally, all experiments are performed on an NVIDIA A100 system consisting of a single node with 4 GPUs, each with 40 GB of VRAM.

5.2. Results

Table 4 summarizes the performance results, measured by relative L_2 error, for various models and architectures across different test cases. Table 5 shows the detailed final training losses for each loss term.

5.2.1. Helmholtz equation

For the Helmholtz equation, B-spline(A1) achieves the lowest velocity error (1.93%), followed by the Tanh(A1) (6.13%). In contrast, GRBF(A1) and Jacobi(A1) result in significantly higher errors. KAN models with A2 architecture exhibit instability and failure to solve the Helmholtz equation.

Fig. 2 displays the training loss history for the top-performing architectures from each model group listed in Table 3. The plot indicates that the Tanh(A1), Fourier(A1), and B-spline(A1) models exhibit significantly better convergence than the others, with the B-spline(A1) model achieving the lowest total final loss value. Detailed term loss values can be found in Table 5. The higher training error observed in the physics loss term for this case, as well as the others, reflects the weighting of the loss values discussed in section 4. Nevertheless, the physics loss term remains an essential component, functioning as a regularizer.

Fig. 3 compares the reference solution and the predicted solutions from B-spline(A1) and Fourier(A1) for the Helmholtz equation. The majority of inference errors for both models are concentrated near the boundaries of the domain, with the Fourier model showing more pronounced deviations in these regions.

5.2.2. Klein-Gordon equation

The Tanh(A1) model has the lowest velocity error (1.58%), followed by the B-spline(A1) model (5.23%). Similar to the Helmholtz case, KAN models utilizing the A2 architecture show instability, with errors sur-

Table 5

The final training losses include the physics-informed loss (L_{phy}), the boundary conditions loss (L_{bc}), and the initial conditions loss (L_{ic}). Information about these loss functions can be found in Section 4.

Case		MLP				KAN									
		Tanh		param-Tanh		B-spline		GRBF		Fourier		Chebyshev		Jacobi	
		A1	A2	A1	A2	A1	A2	A1	A2	A1	A2	A1	A2	A1	A2
Helmholtz	L_{phy}	4.4e+00	1.91e+00	2.04e+00	2.4e-01	6.1e-01	2.75e-01	1.04e+01	2.7e+00	1.0e+00	3.30e+12	8.3e-01	7.15e+03	2.9e+00	6.80e+03
	L_{bc}	6.6e-03	3.27e-03	6.55e-03	3.1e-03	1.5e-03	1.66e-03	1.30e-02	2.1e-02	2.7e-03	3.39e-01	2.8e-03	1.13e-01	2.3e-02	1.50e-01
Klein-Gordon	L_{phy}	1.9e-01	8.57e+00	1.3e+00	1.31e+00	3.0e-01	2.01e-01	1.2e-01	2.64e-01	4.6e-01	2.29e+06	2.4e+00	1.19e+04	3.5e-01	9.46e+03
	L_{bc}	6.0e-05	3.86e-04	7.4e-04	7.39e-04	4.2e-05	1.00e-05	2.5e-05	4.74e-04	5.7e-04	5.96e-01	8.6e-04	4.16e-02	3.9e-04	5.21e-02
	L_{ic}	4.5e-06	4.74e-04	7.5e-05	7.47e-05	4.8e-06	4.34e-06	6.1e-06	4.83e-04	8.1e-05	3.30e-01	6.6e-05	2.32e-01	4.3e-04	1.82e-01
Wave	L_{phy}	1.47e-01	1.2e-01	3.24e-02	3.7e-02	3.1e-02	3.00e-02	8.83e-01	6.6e-01	1.3e+00	1.65e+10	2.9e-01	3.77e-01	8.9e-01	1.89e-07
	L_{bc}	1.46e-02	4.3e-04	1.09e-02	1.7e-04	5.0e-05	2.74e-05	1.45e-03	1.6e-03	1.3e-02	9.30e-02	1.5e-02	3.50e-02	1.8e-02	5.97e-02
	L_{ic}	3.54e-03	7.8e-05	4.59e-03	3.6e-05	7.6e-06	9.23e-06	6.64e-04	5.8e-04	1.3e-02	3.92e-01	3.2e-03	1.15e-02	7.7e-03	4.50e-01
Convection Diffusion	L_{phy}	7.2e-04	1.74e-03	2.74e-04	5.8e-03	6.0e-05	5.30e-05	2.7e-04	7.28e-03	6.3e-03	5.52e+05	1.1e-03	1.02e+00	8.5e-04	9.69e-01
	L_{bc}	3.0e-05	3.16e-05	2.15e-05	2.6e-05	8.3e-06	2.28e-05	2.3e-05	4.39e-05	1.3e-04	4.50e-03	1.6e-04	1.02e-02	8.6e-04	3.67e-05
	L_{ic}	5.4e-05	8.99e-05	7.77e-05	1.2e-04	5.3e-06	4.31e-06	6.3e-05	7.21e-05	1.3e-04	1.37e-02	1.2e-04	8.87e-03	4.2e-04	1.61e-02
Cavity	L_{phy}	1.42e-01	2.2e-02	1.3e-01	2.79e-02	2.56e-01	2.3e-02	1.4e-01	6.75e-02	2.1e-01	3.25e-01	1.34e-01	2.9e-02	2.4e-01	1.86e-07
	L_{bc}	1.35e-02	4.5e-04	3.4e-04	1.08e-02	2.95e-03	1.5e-05	4.4e-03	5.06e-03	3.6e-03	1.50e-02	1.39e-02	1.4e-03	9.1e-03	1.02e+00
	L_{ic}	2.57e-03	6.6e-03	4.0e-03	3.81e-03	6.53e-04	2.5e-04	5.7e-03	4.57e-03	7.6e-03	1.04e-02	3.81e-03	3.7e-03	6.7e-03	8.42e-02

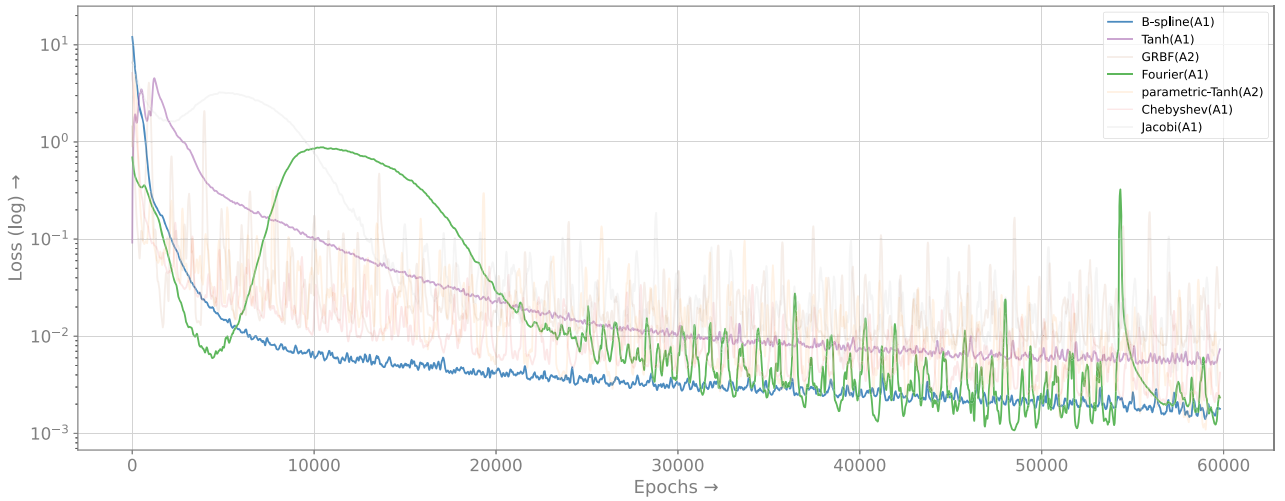


Fig. 2. Training convergence history for the Helmholtz equation. The loss curves for all models are shown, with **B-spline(A1)**, **Fourier(A1)**, and **Tanh(A1)** highlighted in saturated colors, which demonstrated the best convergence behavior. The remaining models are depicted with transparency to enhance visual distinction and clarity. (For interpretation of the colors in the figure(s), the reader is referred to the web version of this article.)

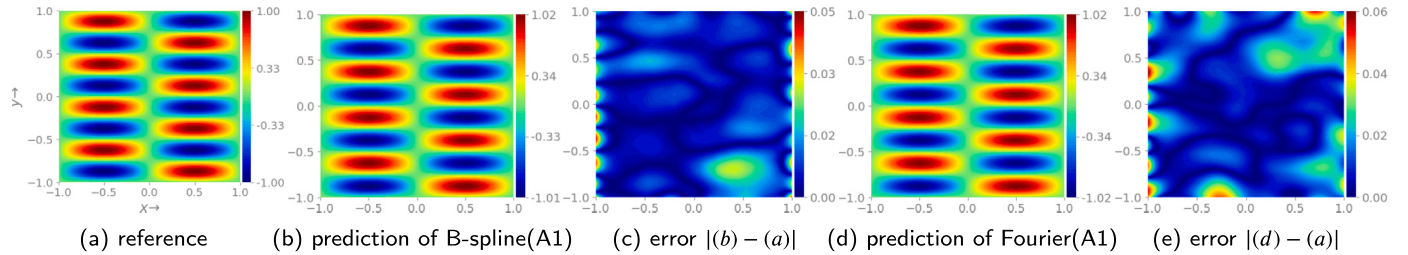


Fig. 3. Contour plots for the solution of the Helmholtz equation. The plots depict the reference or actual solution, the predicted solution, and the corresponding error for the two best-performing models identified in Table 4. (b) presents the prediction of **B-spline(A1)** model, which achieves the lowest error of 1.93%, while (d) shows the prediction of **Fourier(A1)** model, which has a relative error of 3.09%.

passing 90.0% in Fourier, Chebyshev, and Jacobi, indicating a failure in performance.

Fig. 4 illustrates the training convergence history for the Klein-Gordon equation. The Tanh(A1) model attained the lowest relative error, while the B-spline(A1) exhibited a more stable convergence with minimal fluctuations. The models stabilize with total training errors of approximately 0.19 for Tanh(A1) and 0.30 for B-spline(A1), as shown

in Table 5. In contrast, other models exhibited instability and oscillatory behavior, indicating difficulties in optimizing the solution under the specified settings.

Fig. 5 presents the solution of the Klein-Gordon equation, comparing the reference solution to the best-predicted solutions from Tanh(A1) and B-spline(A1), along with their corresponding error plots. Both models exhibit error concentration in regions with high gradients, where the

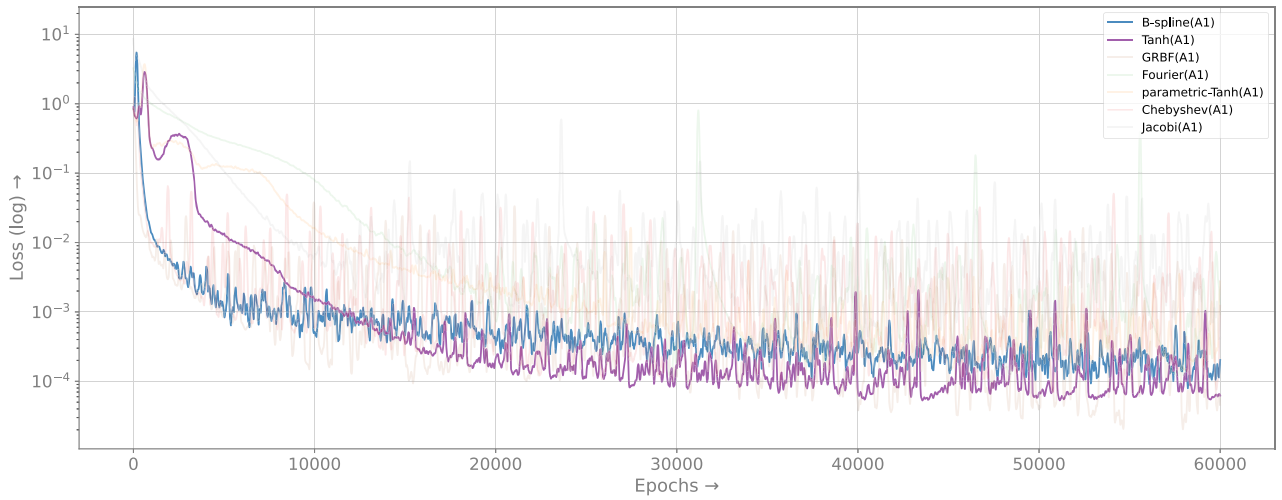


Fig. 4. Training convergence history for the Klein-Gordon equation. The loss curves for the two best converging models, **Tanh(A1)** and **B-spline(A1)**, are highlighted.

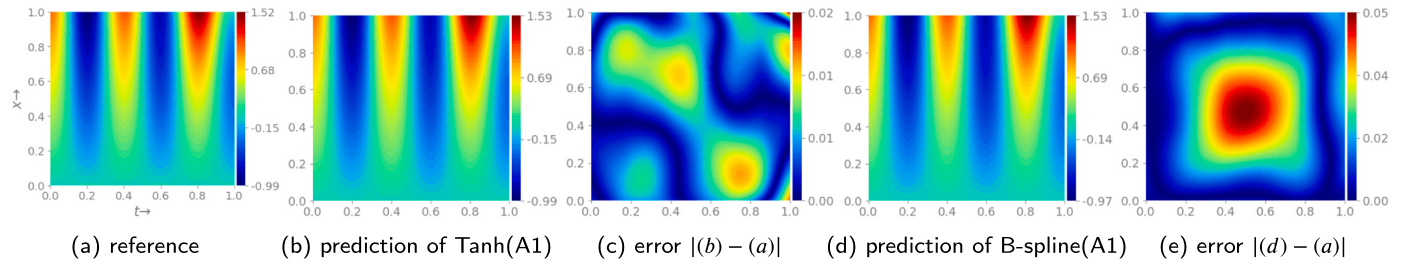


Fig. 5. Contour plots for the solution of the Klein-Gordon equation. The plots depict the reference or actual solution, the predicted solution, and the corresponding error for the two best-performing models identified in Table 4. (b) presents the prediction of **Tanh(A1)** model, which achieves the lowest error of 1.58%, while (d) shows the prediction of **B-spline(A1)** model, which has a relative error of 5.23%.

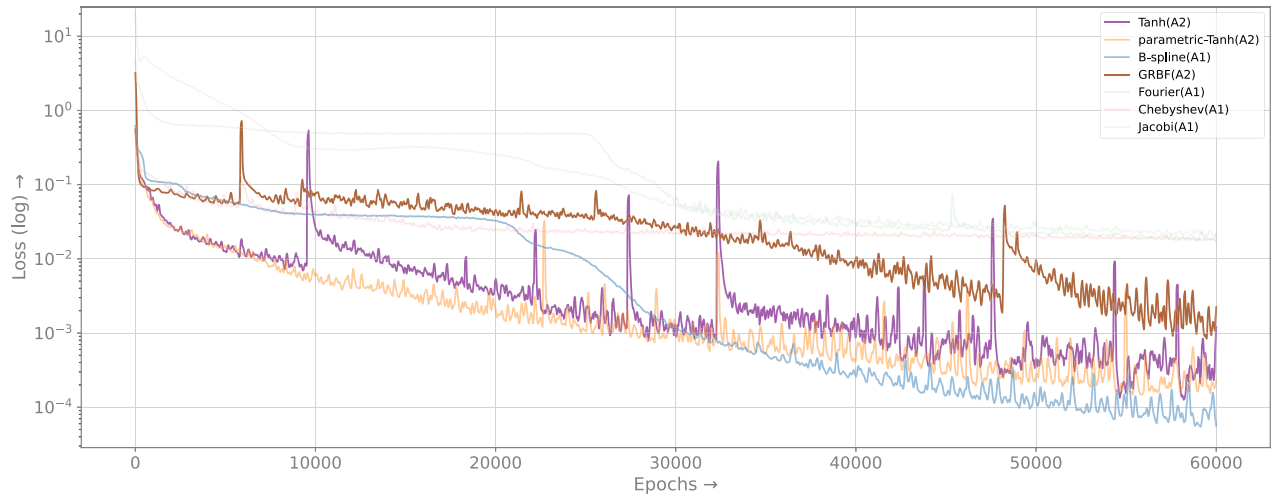


Fig. 6. Training convergence history for the Wave equation. The loss curves indicate that **B-spline(A1)** and **parametric-Tanh(A2)** models exhibit the most consistent convergence. However, **Tanh(A2)** and **GRBF(A2)** models achieve the lowest testing errors, as reported in Table 4.

solution changes rapidly. However, the Tanh(A1) handles these high-gradient areas more effectively, as indicated by the lower overall error in Fig. 5(c) compared to the error observed for the B-spline results in Fig. 5(e).

5.2.3. Wave equation

The Tanh(A2) model achieves the lowest velocity error (13.6%), followed by GRBF(A2) (14.7%) (as in Table 4). Fig. 6 presents the training convergence history for the Wave equation. The plots indicate

that B-spline(A1), Tanh(A2), and GRBF(A2) demonstrate better convergence. The B-spline(A1) model experienced a slower convergence rate during the initial half of the training period, after which it began to converge more rapidly, ultimately reaching the lowest loss value. The parametric-Tanh(A2) showed steady convergence, while both Tanh(A2) and GRBF(A2) exhibited some fluctuations throughout the training process.

Fig. 7 presents the contour plot predictions for the velocity fields using the Tanh(A2) and GRBF(A1) models. The Tanh(A2) model ef-

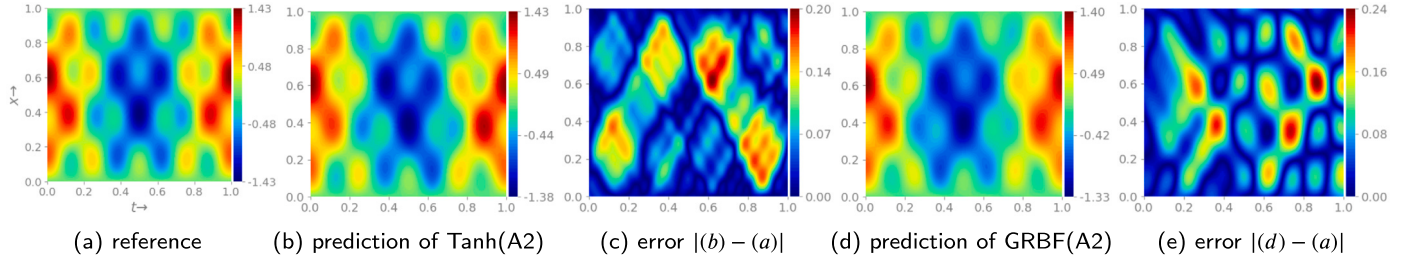


Fig. 7. Contour plots for the solution of the Wave equation in the spatio-temporal domain x . The plots depict the reference or actual solution, the predicted solution, and the corresponding error for the two best-performing models identified in Table 4. (b) presents the prediction of **Tanh(A2)** model, which achieves the lowest error of 13.6%, while (d) shows the prediction of **GRBF(A2)** model, which has a relative error of 14.7%.

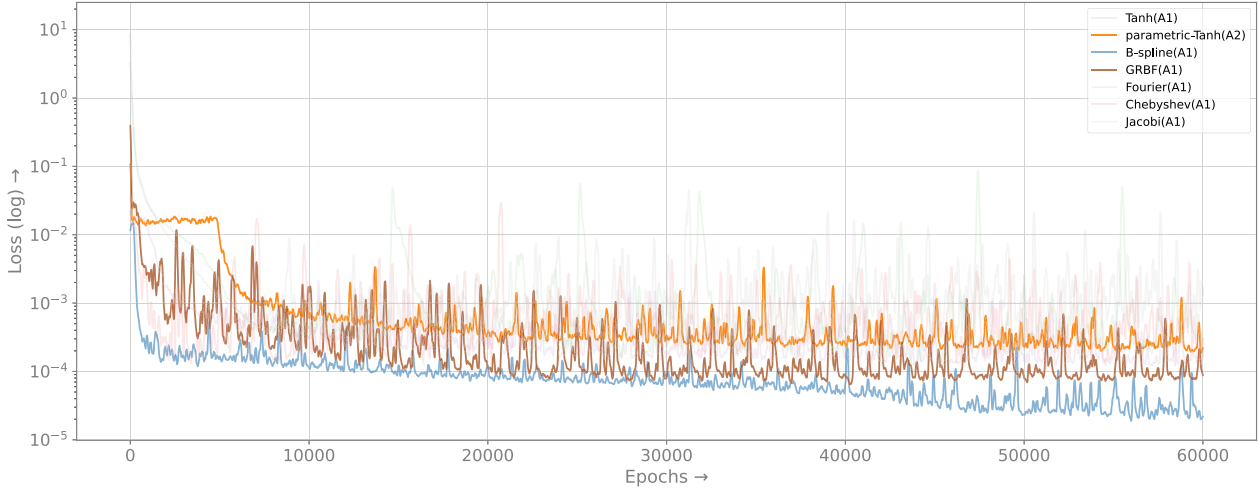


Fig. 8. Training convergence history for the Convection-diffusion equation. The loss curves for all models are displayed, with **B-spline(A1)**, **GRBF(A1)**, and **parametric-Tanh(A2)** demonstrated the best convergence.

fectively captures the continuous dynamics of the Wave equation. In contrast, the GRBF(A1) model exhibits a slightly higher relative error compared to Tanh(A2), with most of the errors occurring in areas where the wave oscillates rapidly.

5.2.4. Convection-diffusion equation

The GRBF(A1) model achieves the lowest velocity u followed by parametric-Tanh(A2) (refer to Table 4). In contrast, Fourier, Chebyshev, and Jacobi models exhibited high errors with the A1 architecture and failed to converge effectively with A2.

Fig. 8 presents the training convergence history for the Convection-diffusion equation. The plots indicate that B-spline(A1), GRBF(A1), and parametric-Tanh(A2) exhibit better convergence compared to other models. The B-spline(A1) model shows the most consistent convergence and the lowest training loss across both architectures, as illustrated in Table 5. In contrast, the GRBF(A1) model shows less stability and more noticeable fluctuations. The remaining models display considerable instability. Although B-spline(A1) achieves the fastest convergence and lowest final loss value, the test results, as summarized in Table 4, indicate that GRBF(A1) and parametric-Tanh(A2) deliver lower relative errors.

Fig. 9 presents the solution of the Convection-Diffusion equation for the velocity and the force values at the beginning and the end of the simulation using parametric Tanh(A2) and the GRBF(A1) models. The reference velocity is concentrated at the center (0.5, 0.5) at the initial time step (initial condition), dissipating as time progresses (as shown by the shrinking central area at time step 1). Similarly, the force field is shown in the third and fourth columns, with the high-intensity areas at time-step zero diffusing over time.

In the second row of Fig. 9, the GRBF model captures the dissipation of the velocity field reasonably well, with only minor errors (as seen in

the error plot in row (c)). Similarly, for the force field, the GRBF model captures the general diffusion pattern accurately, with relatively low errors (compared to other models). Parametric Tanh(A2) has a slightly higher relative error for velocity than GRBF(A1).

5.2.5. Cavity equation

The results in Table 4 show that the B-spline(A2) model achieves the lowest errors, followed by Tanh(A2). In contrast, the Fourier, Chebyshev, and Jacobi perform less effectively with errors exceeding 30% across all models.

Fig. 10 shows training convergence history for the Cavity problem. The B-spline(A2) model demonstrates the most effective convergence, starting from a relatively high initial loss but rapidly decreasing. However, it exhibits relatively unstable behaviors, characterized by persistent fluctuations throughout the training process. Ultimately, B-spline(A2) achieves the lowest final training loss value, followed by Tanh(A2) after 60,000 iterations.

Fig. 11 presents the solution of the Cavity problem, comparing the reference solution to predictions made by B-spline(A2) and Tanh(A2) at the initial Cavity flow. The predicted u field by the Tanh(A2) is mostly close to zero, but there are slight deviations at the bottom. However, the v field shows deviations from the reference. It exhibits sharp gradients along the top boundary. The smooth nature of the Tanh activation seems to struggle to capture the sharp boundary behavior accurately. The pressure field predicted by the Tanh(A2) has significant deviations, particularly near the top boundary. There are some deviations in the B-spline(A2) model, but the overall structure is closer to the reference solution.

Fig. 12 shows the plots of the reference solution, the Tanh, and the B-spline approximations when solving the Cavity problem at the steady state. The Tanh(A2) model reasonably approximates u field, showing

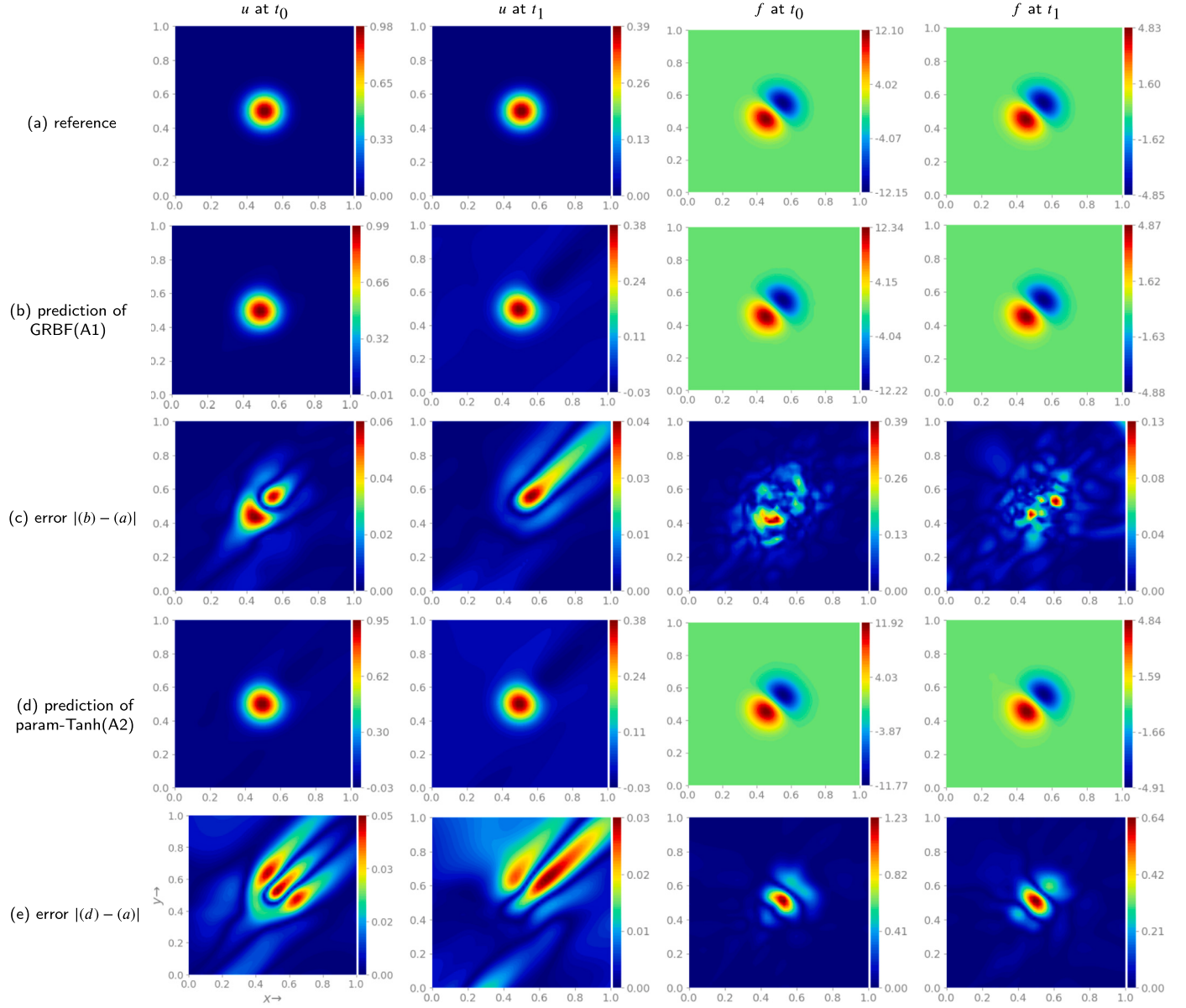


Fig. 9. Contour plots for the solution of the Convection-diffusion velocity (u) and force (f) values at time-step zero (t_0) and one (t_1) in the spatio-temporal domain. The plots at each row depict the reference or actual solution, the predicted solution, and the corresponding error for the two best-performing models identified in Table 4, respectively. (b) presents the prediction of GRBF(A1) model, the relative L_2 errors are 11.5% 2.07% for u and f , respectively. (d) presents the prediction of parametric-Tanh(A2) model, the relative L_2 errors are 11.6% 1.07% for u and f , respectively.

a similar pattern to the reference solution with high velocities near the top boundary. However, the velocity distribution near the bottom of the cavity has more deviation from the reference. The v field shows noticeable differences compared to the reference. The pressure field predicted by the Tanh(A2) model shows a significant deviation from the reference, especially at the top boundary.

5.2.6. Computational complexity

Table 4 shows the time required for training in seconds per iteration for each model. The number of parameters for MLP models is much lower than that of KAN models for basis functions (Table 3) and possibly extra weights for handling those functions and their derivatives for propagation and physics loss term calculations (Table 2). MLP with Tanh activation functions, followed by GRBF, generally have lower times per iteration (t/it), while KAN with Jacobi basis function tends to have the highest computational cost compared to other models. This increased computational cost for Jacobi can be attributed to its complex polynomial formulation, which requires evaluating higher-degree terms (up to

$d = 4$) with multiple coefficients that scale with the polynomial degree as shown in Table 2.

The relatively small increase in time per iteration, despite the larger parameter count in architecture A2 compared to architecture A1 within the same model, is likely a result of the NVIDIA A100's parallel processing capabilities and efficient utilization of optimizations in PyTorch. Furthermore, 128 batch size balances computational efficiency and memory usage. Since the same minibatch size is used for both A1 and A2 architectures, the increased parameter count in A2 may not be sufficient to cause significant changes in time per iteration.

6. Spectral bias and convergence analysis

In this section, we analyze the behavior of the eigenvalues of the NTK matrix as a spectral bias measurement and the maximum Hessian eigenvalue of the loss function with respect to neural network parameters as a convergence measurement tool using the best-performing models from the previous sections. These experiments employ the Adam optimizer,

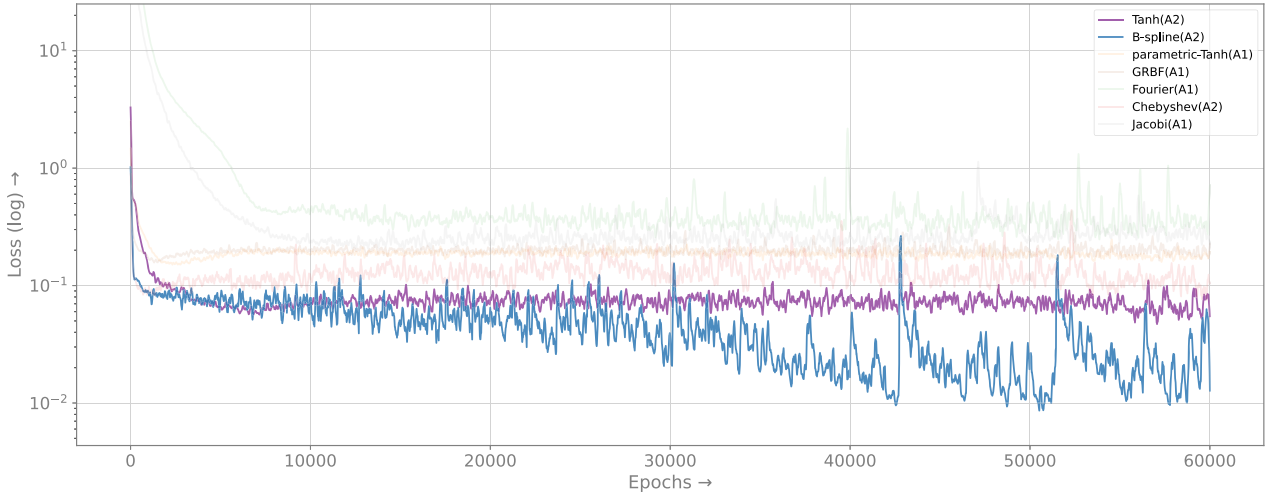


Fig. 10. Training convergence history for the Cavity problem. Loss curves for all models are shown, with **B-spline(A2)** and **Tanh(A2)**, which demonstrated the best convergence.

configured with a learning rate of 0.001 and a weight decay of 10^{-6} . Additionally, mini-batch gradient descent is employed with a batch size of 128 for a total of 60,000 iterations. To better understand the behavior of KAN models, only the basic basis function is used; additional activation functions, such as SiLU from Eq. (4), have been omitted.

6.1. Spectral bias with NTK

The empirical NTK offers an approximate theoretical framework for understanding the dynamics of neural network training and spectral bias. For a neural network with parameters θ , the empirical NTK matrix between inputs $\mathbf{x}$ and $\mathbf{x}'$ at time t is defined as [46,65]:

$$K_t(\mathbf{x}, \mathbf{x}') = \nabla_{\theta} f(\mathbf{x}; \theta_t) \cdot \nabla_{\theta} f(\mathbf{x}'; \theta_t)^{\top} \quad (21)$$

In the context of PINNs, we compute the empirical NTK matrix for batches of inputs $\mathbf{x}_1 = \{\mathbf{x}_1^{(1)}, \dots, \mathbf{x}_1^{(m)}\}$ and $\mathbf{x}_2 = \{\mathbf{x}_2^{(1)}, \dots, \mathbf{x}_2^{(n)}\}$. We construct the Jacobian matrices $\mathbf{J}_1 \in \mathbb{R}^{n \times |\theta|}$ and $\mathbf{J}_2 \in \mathbb{R}^{n \times |\theta|}$ where:

$$[\mathbf{J}_1]_{i,j} = \frac{\partial f(\mathbf{x}_1^{(i)}; \theta)}{\partial \theta_j}, \quad [\mathbf{J}_2]_{i,j} = \frac{\partial f(\mathbf{x}_2^{(i)}; \theta)}{\partial \theta_j} \quad (22)$$

The empirical NTK matrix is then computed as:

$$\mathbf{K} = \mathbf{J}_1 \mathbf{J}_2^{\top} \in \mathbb{R}^{n \times n} \quad (23)$$

Following the NTK theory framework [24,33], the training dynamics in the infinite-width limit can be characterized by analyzing the eigenvalue decomposition of $\mathbf{K}$:

$$\mathbf{K} = \mathbf{Q} \mathbf{\Lambda} \mathbf{Q}^{\top} \quad (24)$$

assuming $\mathbf{K}$ is positive semi-definite from Eq. (23); where, $\mathbf{\Lambda} = \text{diag}(\lambda_1, \lambda_2, \dots, \lambda_n)$ contains the eigenvalues in descending order $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_n \geq 0$ and $\mathbf{Q}$ is orthogonal matrix. Since $e^{-\mathbf{K}t} = \mathbf{Q}^{\top} e^{-\mathbf{\Lambda}t} \mathbf{Q}$, the eigenvalue spectrum directly governs the learning dynamics through the exponential convergence rates. Under gradient descent with learning rate η , the network function evolves according to:

$$\begin{aligned} \mathcal{L}(\theta) &= \frac{1}{2} \|f(\mathbf{x}; \theta) - f^*(\mathbf{x})\|^2 \\ \frac{d\theta}{dt} &= -\eta \nabla_{\theta} \mathcal{L}(\theta) \\ \nabla_{\theta} \mathcal{L} &= \nabla_{\theta} f(\mathbf{x}; \theta) \cdot (f(\mathbf{x}; \theta) - f^*(\mathbf{x})) \\ \frac{df(\mathbf{x}; \theta)}{dt} &= \nabla_{\theta} f(\mathbf{x}; \theta) \cdot \frac{d\theta}{dt} \end{aligned}$$

$$\frac{df(\mathbf{x}; \theta)}{dt} = -\eta \nabla_{\theta} f(\mathbf{x}; \theta) \cdot \nabla_{\theta} \mathcal{L}$$

$$\frac{df(\mathbf{x}; \theta)}{dt} = -\eta \nabla_{\theta} f(\mathbf{x}; \theta) \cdot \nabla_{\theta} f(\mathbf{x}'; \theta) \cdot (f(\mathbf{x}'; \theta) - f^*(\mathbf{x}'))$$

$$\frac{df(\mathbf{x}; \theta)}{dt} = -\eta \sum_{\mathbf{x}'} K(\mathbf{x}, \mathbf{x}') (f(\mathbf{x}'; \theta) - f^*(\mathbf{x}'))$$

$$\frac{df(t)}{dt} = -\eta \nabla_{\theta} \mathcal{L}(\theta) \cdot \nabla_{\theta} f = -\eta \mathbf{K} (f(t) - f^*)$$

This is a first-order PDE with the solution:

$$f(t) - f^* = e^{-\eta \mathbf{K}t} (f(0) - f^*) \quad (25)$$

Using Eq. (24) for $\mathbf{K}$, yields:

$$[\mathbf{Q}^{\top} (f(t) - f^*)]_i \approx e^{-\eta \lambda_i t} [\mathbf{Q}^{\top} (f(0) - f^*)]_i \quad (26)$$

where, $f(t)$ represents the network predictions at iteration t , f^* is the target function, η is the learning rate, and $f(0)$ is the initial condition. This reveals that eigenfunction components with larger eigenvalues λ_i converge exponentially faster at rate $\eta \lambda_i$.

For KAN networks with learnable basis function coefficients c_{ijr} , the NTK analysis can be extended:

$$K_t^{\text{KAN}}(\mathbf{x}, \mathbf{x}') = \sum_{i,j,r} \frac{\partial f(\mathbf{x})}{\partial c_{ijr}} \frac{\partial f(\mathbf{x}')}{\partial c_{ijr}} \bigg|_{\theta_t} \quad (27)$$

Thus, the choice of the basis function significantly influences the NTK eigenvalue spectrum and resulting spectral bias. However, the user-defined variables, such as grid size and basis function order, can make the analysis less straightforward.

In general, it is proven that Fourier-based activations modify the NTK to achieve an improved high-frequency representation, resulting in eigenvalue spectra with slower decay [60,79]. For GRBF kernels, higher eigenvalues correspond to low-frequency (smooth) functions, while lower eigenvalues correspond to high-frequency (oscillatory) functions [24,46]. B-spline and polynomial activations exhibit intermediate behavior, with eigenvalue distributions that depend on the order of the basis function and the domain coverage [25,36,79].

Fig. 13 shows the evolution of the NTK eigenvalue spectra in descending order for the initial (top), middle (middle), and end (bottom) of training iterations. The figure reveals that even with adaptive basis functions, spectral bias persists to different degrees. The NTK spectra show that the training dynamics evolve in a problem-dependent manner. At initialization (top row), the Fourier and Jacobi activations consistently exhibit the flattest eigenvalue decay across all problems, indicating their

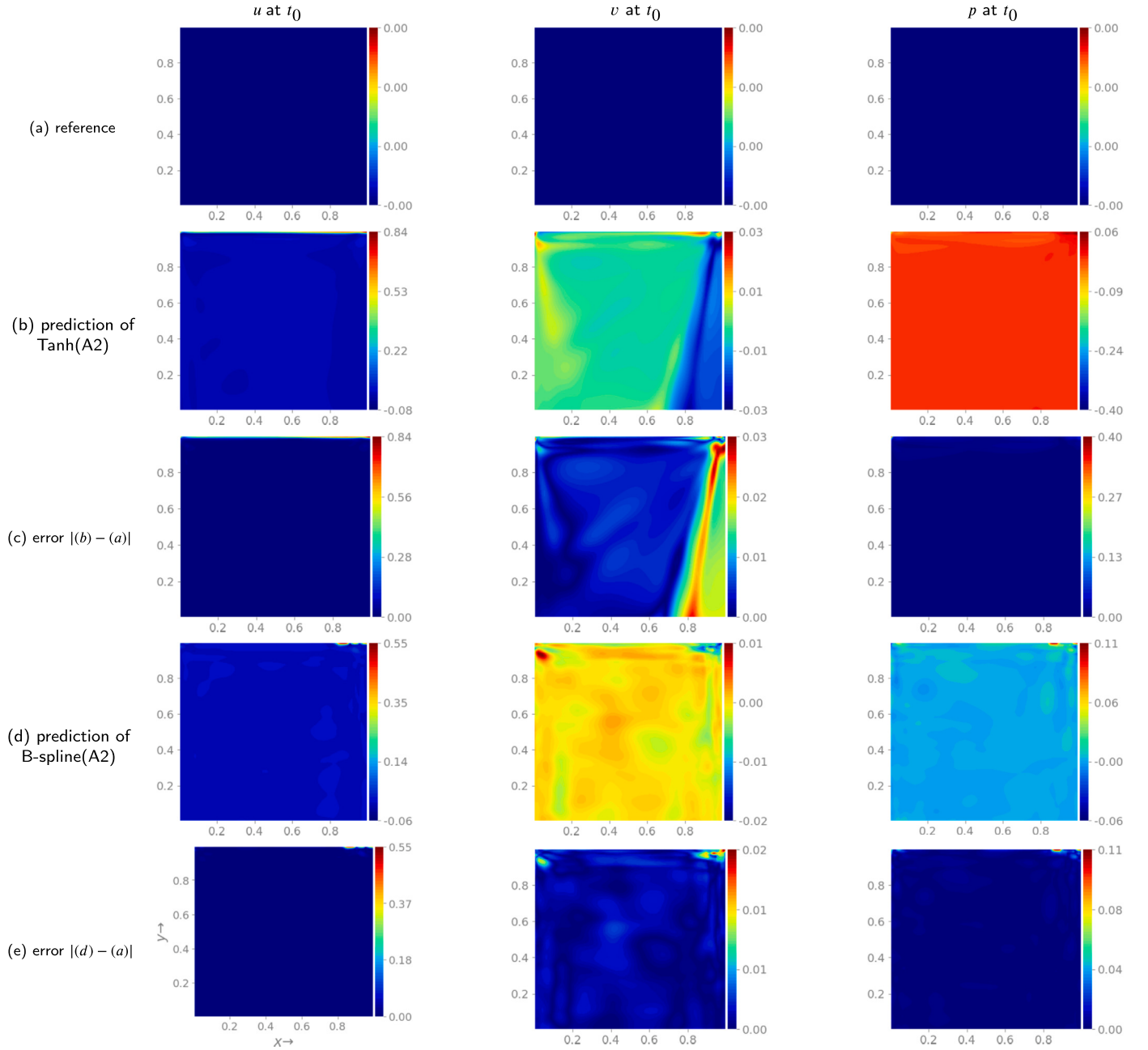


Fig. 11. Contour plots for the solution of the Cavity problem velocity (u and v) and pressure (p) values at the initial condition (t_0), with $Re = 100$. (b) presents the prediction of **Tanh(A2)** model, the relative L_2 errors as 10.8%, 22%, and 24.4% for u , v , and p , respectively. (d) presents the prediction of **B-spline(A2)** model, the relative L_2 errors are 4.1%, 8.7%, and 21.3% for u , v , and p , respectively.

capacity to represent high-frequency modes. In contrast, B-spline and Tanh activations show steep initial decay. As training progresses (middle and bottom rows), all models experience varying degrees of spectral compression, reflecting a gradual collapse of higher NTK eigenvalues. However, Fourier maintains a broader spectrum throughout training. While the NTK spectrum suggests that B-spline-based activations exhibit high spectral bias, they perform remarkably well on testing data. This can be understood by drawing an analogy to finite basis PINNs (FBPINNs) [44,16], where domain decomposition ensures that globally high-frequency content appears locally low-frequency. Similarly, B-splines, being piecewise-defined with local support, can capture local oscillations, circumventing global spectral limitations.

Examining the NTK eigenvalues (plots from left to right in Fig. 13) reveals essential insights into the learning dynamics across loss components. For the boundary condition domain, most activations show

rapid eigenvalue decay due to the inherent smoothness of boundary enforcement. The initial condition domain presents an intermediate decay profile, balancing smoothness with the need to capture initial transients. The physics residual domain consistently exhibits the broadest NTK spectra, primarily due to its larger dataset size compared to the boundary and initial components. Notably, activations such as Fourier and Jacobi consistently retain higher eigenvalues in the physics domain, ensuring their capacity to capture high-frequency features and complex solution structures that arise within the domain.

In terms of training dynamics, the NTK eigenvalue spectra initially reflect the inherent biases of each activation architecture. As training progresses, most activations, except Fourier, demonstrate progressive spectral compression, indicating a collapse towards lower frequencies. Notably, Fourier activations maintain a broader spectrum throughout training.

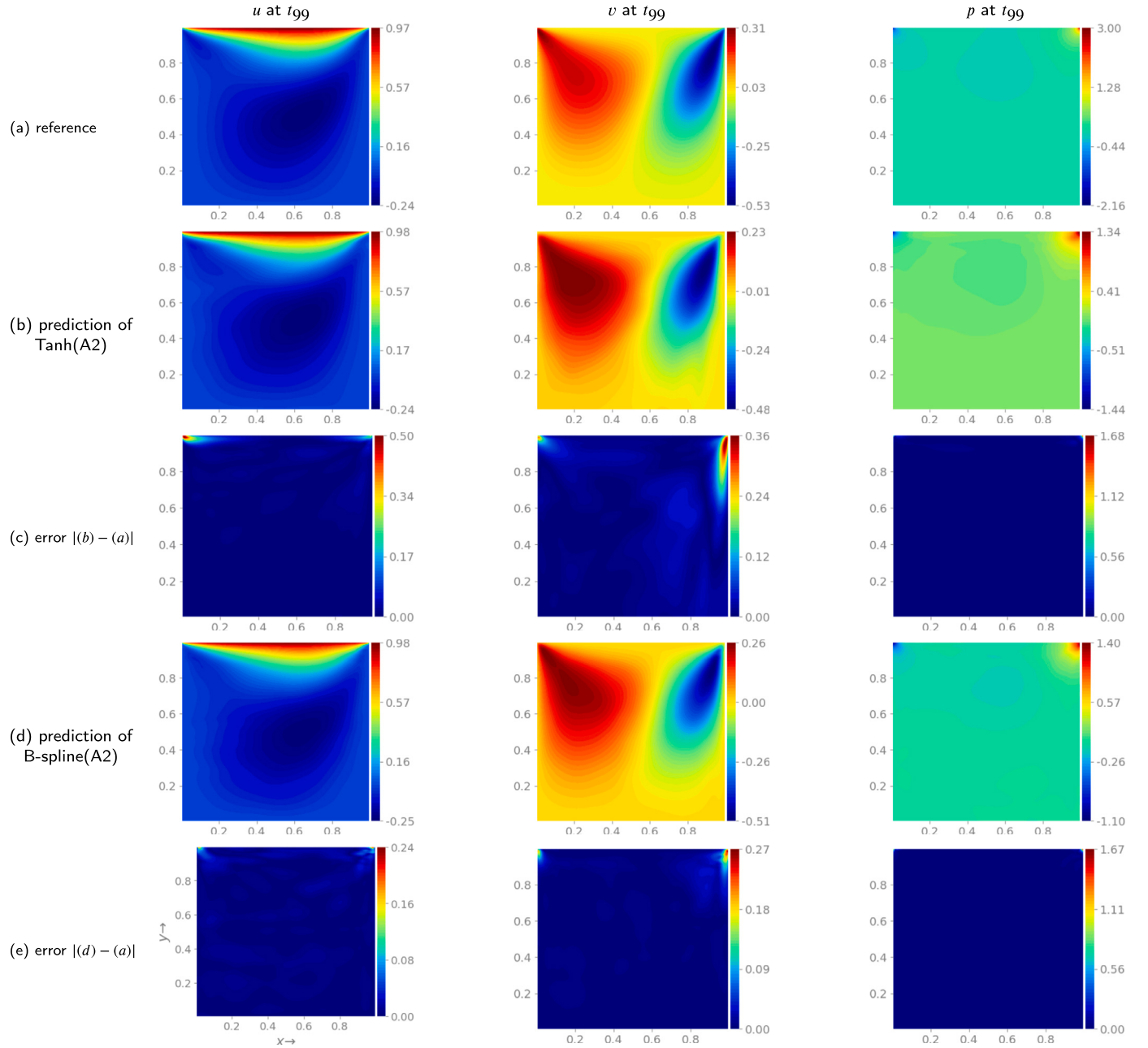


Fig. 12. Contour plots for the solution of the Cavity problem, showing velocity components (u and v) and pressure (p) values at time-step 99 (t_{99}), with $Re = 100$. (b) shows the prediction from the **Tanh(A2)** model, with relative L_2 errors of 10.8% 22% 24.4% for u , v , and p , respectively. (d) shows the prediction from the **B-spline(A2)** model, which achieved relative L_2 errors of 4.1%, 8.7%, 21.3% for u , v , and p , respectively. Note that only the result of t_0 is recorded in Table 4, results of t_{99} can be reproduced using the pretrained model provided with our source code.

6.2. Convergence with loss Hessian

We extend the NTK analysis by examining the maximum eigenvalue of the Hessian matrix $\mathbf{H}$ to analyze the convergence behavior of the models [19,12,1,37,18,29]. We utilized the best-performing models discussed in Section 5.2. To minimize the computation overhead, we estimate the largest eigenvalue $\lambda_{\max}$ of the Hessian matrix as a Rayleigh quotient using the power iteration method [43,74]:

$$\lambda_{\max} = \lim_{k \rightarrow \infty} \frac{\mathbf{v}^T \mathbf{H}^k \mathbf{v}}{\mathbf{v}^T \mathbf{H}^{k-1} \mathbf{v}} \quad (28)$$

where, k is the iteration number, $\mathbf{H}$ is the Hessian matrix, and $\mathbf{v}$ is a random initial vector that converges to the eigenvector corresponding to the maximum eigenvalue.

Fig. 14 presents the maximum eigenvalue of the Hessian of the loss function, denoted as $\lambda_{\max}(\nabla_{\theta}^2 \mathcal{L})$. The B-spline basis function consistently exhibits lower and more stable values of $\lambda_{\max}$, which indicates a smoother optimization process. This observation is supported by the results discussed earlier, where B-splines achieve low error rates and demonstrate steady convergence across various test cases.

The GRBF basis functions initially exhibit some instability as indicated by the fluctuating $\lambda_{\max}$, but eventually stabilize. While the Fourier kernel functions are naturally well-suited for representing periodic, high-frequency components, the observed fluctuations in $\lambda_{\max}$ indicates some optimization challenges due to sharp curvatures in the loss landscape. Similarly, Chebyshev and Jacobi basis functions show significant instability, as reflected in their fluctuating $\lambda_{\max}$ values. This instability is confirmed by their higher error rates and unstable conver-

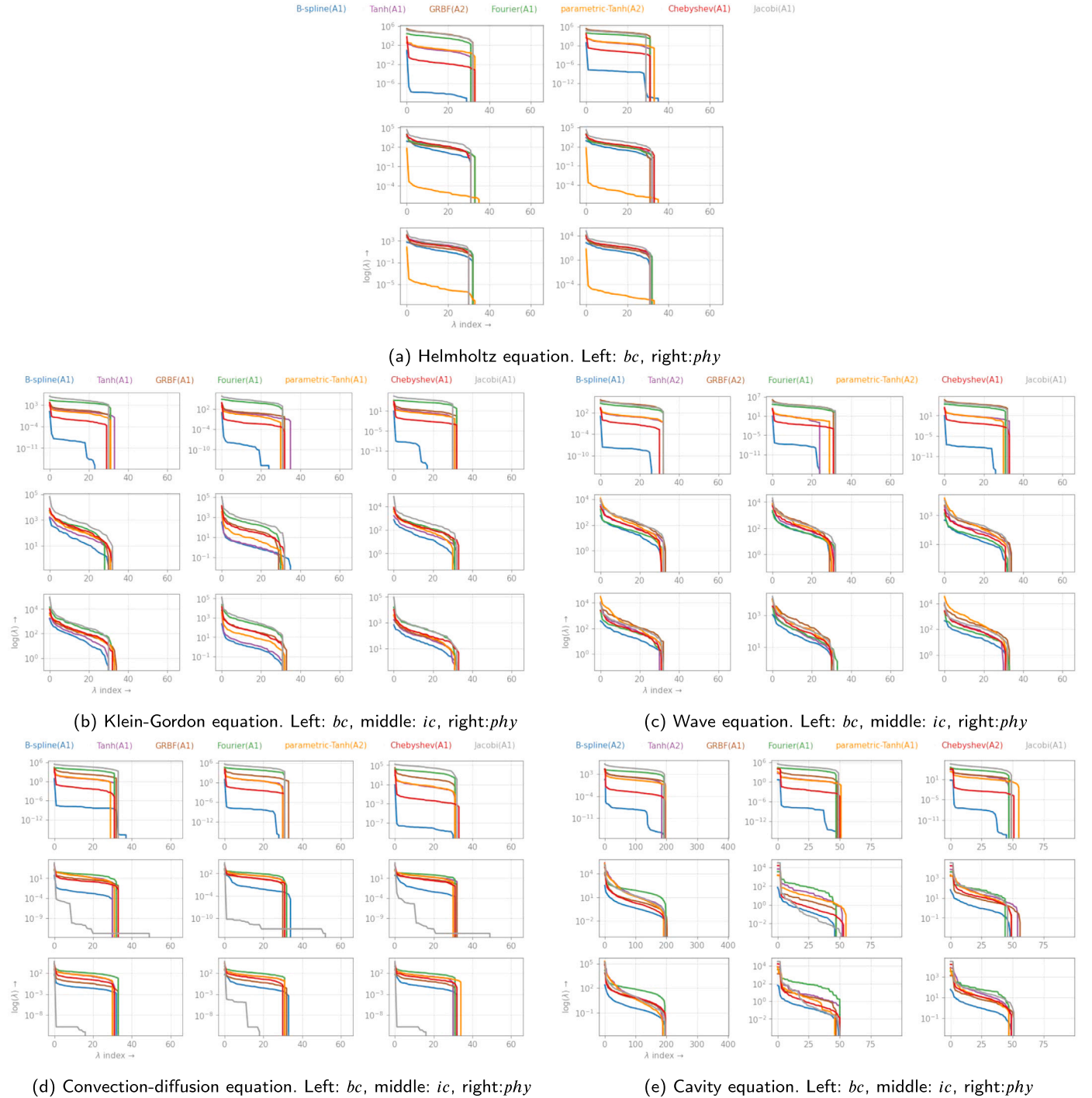


Fig. 13. Evolution of the NTK eigenvalue spectra during training, including the zero eigenvalues. Each row of a subplot corresponds to different training iterations: the initial (top), middle, and final (bottom) stages of the training epochs. The horizontal axis of each plot represents the eigenvalue index, and the vertical axis ($\log(\lambda)$) represents the log eigenvalue.

gence seen in the loss history figures. In the case of the Jacobi basis function, the significant fluctuations in λ_{max} , particularly within the physics loss term, indicate major instability, which correlates with poor performance in several test cases.

Fig. 14 shows that the maximum eigenvalue of the physics loss consistently takes longer to stabilize and is generally larger than the eigenvalues of the boundary and initial condition losses, even for more stable basis functions like B-splines. This discrepancy occurs because the physics loss Hessian captures more complex, high-frequency information and is applied across the entire problem domain, often utilizing

more training datasets than the other terms. Additionally, the physics loss term has lower weighting values compared to the higher weighting values of the boundary and initial loss terms, as discussed in Section 4.

In cases, e.g., Helmholtz, Klein-Gordon, and Cavity problems, the fluctuations in the physics loss Hessian are particularly pronounced. The presence of negative eigenvalues in the physics loss Hessian, as observed in Convection-diffusion and Cavity problems, highlights the challenging optimization landscape for the physics loss compared to the boundary and initial condition losses. The appearance of negative curvature further indicates that gradient-based optimizers, such as Adam, may

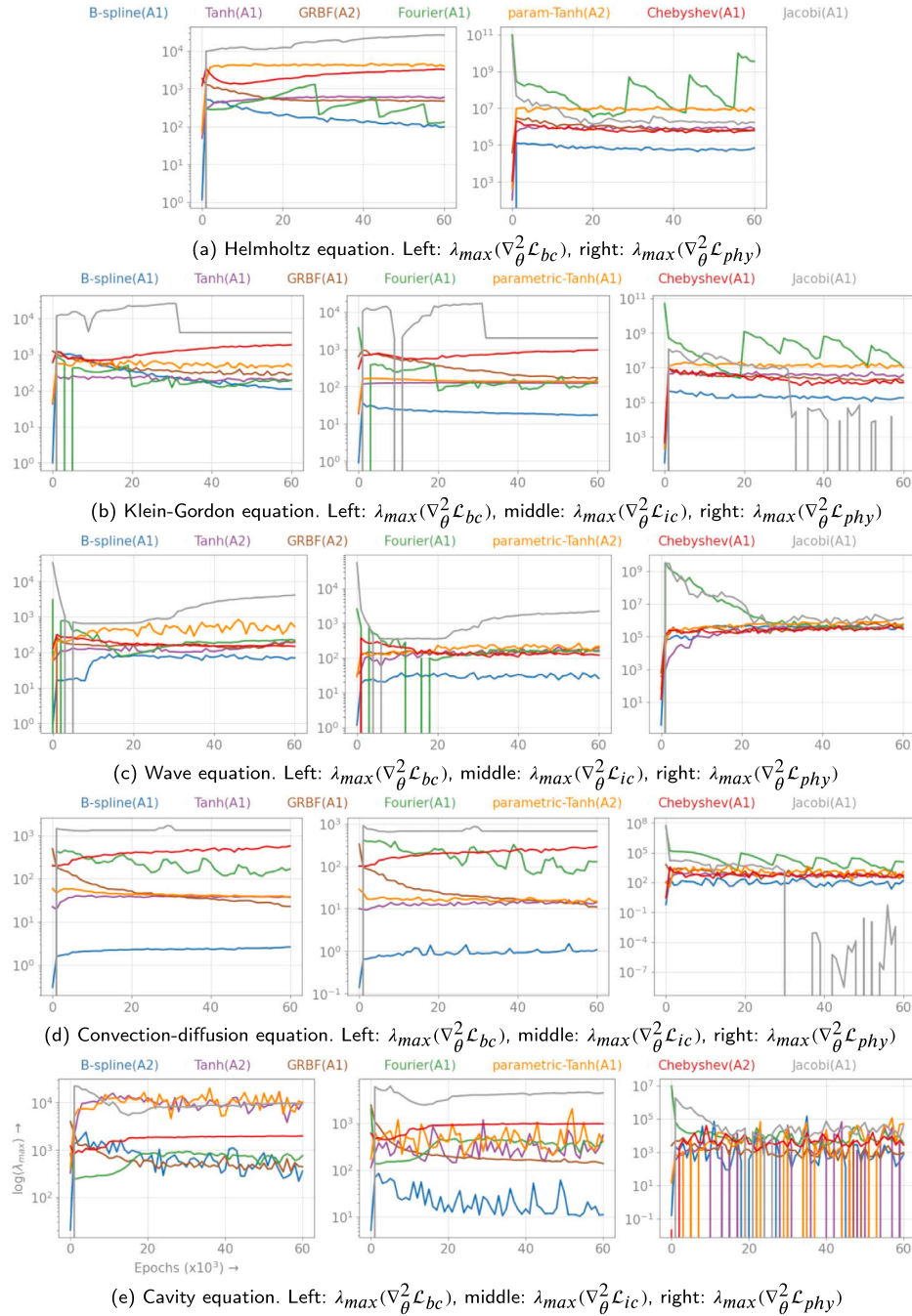


Fig. 14. Evolution of the maximum eigenvalue of the Hessian of the loss function, $\lambda_{\max}$, over training epochs for different neural network models. Large Hessian eigenvalues may indicate optimization difficulties with high frequencies, especially in the physics loss term.

struggle in these cases, as they are better suited for convex optimization problems. Negative curvature can cause optimization algorithms to get trapped in saddle points or lead to slower convergence. Recent studies, such as [49], suggest Newton's method or preconditioned gradient descent techniques can help navigate these regions by using second-order information to escape saddle points and improve convergence rates.

7. Conclusion

Our results show a trade-off between expressivity and training convergence stability in learnable activation functions. While these learnable activation functions demonstrate notable effectiveness in simpler architecture, A1, they face significant challenges when scaled to complex networks, A2. This limitation stems from the curse of functional di-

mensionality inherent to KAN architectures: unlike MLPs that learn feature hierarchies through weight matrices, KANs must learn entire function spaces at each connection. This creates an exponentially expanding optimization landscape that overwhelms gradient-based methods. The architectural complexity ultimately leads to optimization intractability and performance degradation across multiple PDE types. This explains the instability of learnable activations, including the parametric-Tanh function, which introduces only two learnable parameters and experiences similar scaling challenges, albeit to a lesser degree.

Contrary to common intuition, low spectral bias does not necessarily guarantee better testing accuracy. A notable example of this is the Fourier basis function. Our analysis shows that while Fourier and Jacobi basis functions maintain broader NTK eigenvalue spectra throughout the training, they simultaneously exhibit the largest maximum Hessian

eigenvalues in physics residual and boundary components, suggesting a potential convergence instability. Conversely, B-splines display rapid NTK eigenvalue decay, indicating high spectral bias. However, B-spline with SiLU maintain high-frequency representation within localized support regions, effectively capturing sharp gradients without introducing global high-frequency interference.

Our findings emphasize the significance of problem-specific activation selection. The Tanh activation function is particularly effective for wave-like PDEs, such as the Klein-Gordon and Wave equations. Surprisingly, the parametric-Tanh fails on the Wave equation. This underscores that activation function selection remains inherently problem-dependent. B-splines prove particularly effective for PDEs with localized features, leveraging their piecewise polynomial construction to model sharp gradients and boundary layers, as demonstrated in the Cavity flow problem. In contrast, Fourier bases struggle with sharp transitions, producing oscillatory artifacts near discontinuities, highlighting the need for advanced optimization techniques or better hyperparameter tuning.

Beyond activation choices, the relative weighting of physics, boundary, and initial condition loss terms remains critical for training success across all architectures. Our Hessian eigenvalue analysis reveals consistently large and unstable eigenvalues, particularly in physics loss terms, indicating that modifications to activation and basis functions alone cannot resolve the fundamental optimization challenges of PINN loss landscapes. Future research should address these optimization issues while developing theoretical frameworks to predict optimal spectral bias distributions for different PDE classes. In conclusion, this work enhances our understanding of how different architectural choices and activation functions impact the performance of neural PDE solvers.

CRedit authorship contribution statement

Afra Farea: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Resources, Methodology, Investigation, Formal analysis, Conceptualization. **Mustafa Serdar Celebi:** Supervision, Project administration, Funding acquisition, Data curation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgement

We thank The National Center for High Performance Computing of Turkey (UHeM) for providing computing resources under grant number 5010662021. We thank Dr. Saiful Khan from Rutherford Appleton Laboratory, Science and Technology Facilities Council, UK, for his valuable comments and extensive feedback on the manuscript. We also thank Emre Cenk Ersan for generating the Cavity flow problem using Ansys Fluent software.

Data availability

Data will be made available on request.

References

- [1] G. Alain, N.L. Roux, P.A. Manzagol, Negative eigenvalues of the Hessian in deep neural networks, in: International Conference on Learning Representations, 2019.
- [2] S.J. Anagnostopoulos, J.D. Toscano, N. Stergiopoulos, G.E. Karniadakis, Residual-based attention in physics-informed neural networks, *Comput. Methods Appl. Mech. Eng.* 421 (2024) 116805.
- [3] A. Apicella, F. Donnarumma, F. Isgrò, R. Prevete, A survey on modern trainable activation functions, *Neural Netw.* 138 (2021) 14–32.
- [4] A. Arzani, J.X. Wang, R.M. D'Souza, Uncovering near-wall blood flow from sparse data with physics-informed neural networks, *Phys. Fluids* 33 (2021) 071905.
- [5] S. Badia, W. Li, A.F. Martín, Finite element interpolated neural networks for solving forward and inverse problems, *Comput. Methods Appl. Mech. Eng.* 418 (2024) 116505.
- [6] J. Bai, G.R. Liu, A. Gupta, L. Alzubaidi, X.Q. Feng, Y. Gu, Physics-informed radial basis network (pirbn): a local approximating neural network for solving nonlinear partial differential equations, *Comput. Methods Appl. Mech. Eng.* 415 (2023) 116290.
- [7] R. Bischof, M.A. Kraus, Multi-objective loss balancing for physics-informed deep learning, *Comput. Methods Appl. Mech. Eng.* 439 (2025) 117914.
- [8] Z. Bozorgasl, H. Chen, Wav-kan: wavelet Kolmogorov-Arnold networks, *arXiv preprint arXiv:2405.12832*, 2024.
- [9] W.L. Briggs, V.E. Henson, S.F. McCormick, *A Multigrid Tutorial*, SIAM, 2000.
- [10] W. Cai, Z.Q.J. Xu, Multi-scale deep neural networks for solving high dimensional pdes, *Neural Inf. Process. Syst.* (2024).
- [11] M. Choraria, L.T. Dadi, G. Chrysos, J. Mairal, V. Cevher, The spectral bias of polynomial neural networks, in: International Conference on Learning Representations, 2022.
- [12] J.M. Cohen, S. Kaur, Y. Li, J.Z. Kolter, A. Talwalkar, Gradient descent on neural networks typically occurs at the edge of stability, in: International Conference on Learning Representations, 2021.
- [13] S. Cuomo, V.S. Di Cola, F. Giampaolo, G. Rozza, M. Raissi, F. Piccialli, Scientific machine learning through physics-informed neural networks: where we are and what's next, *J. Sci. Comput.* 92 (2022) 88.
- [14] G. Cybenko, Approximation by superpositions of a sigmoidal function, *Math. Control Signals Syst.* 2 (1989) 303–314.
- [15] C. De Boor, On calculating with b-splines, *J. Approx. Theory* 6 (1972) 50–62.
- [16] V. Dolean, A. Heinlein, S. Mishra, B. Moseley, Multilevel domain decomposition-based architectures for physics-informed neural networks, *Comput. Methods Appl. Mech. Eng.* 429 (2024) 117116.
- [17] A. Farea, Learnable activation functions in physics-informed neural networks for solving partial differential equations, <https://doi.org/10.5281/zenodo.15629451>, 2025.
- [18] P. Foret, A. Kleiner, H. Mobahi, B. Neyshabur, Sharpness-aware minimization for efficiently improving generalization, in: International Conference on Learning Representations, 2021.
- [19] B. Ghorbani, S. Krishnan, Y. Xiao, An investigation into neural net optimization via Hessian eigenvalue density, in: International Conference on Machine Learning, in: PMLR, 2019, pp. 2232–2241.
- [20] R. Gnanasambandam, B. Shen, J. Chung, X. Yue, Z. Kong, Self-scalable tanh (stan): multi-scale solutions for physics-informed neural networks, *IEEE Trans. Pattern Anal. Mach. Intell.* 45 (2023) 15588–15603, <https://doi.org/10.1109/TPAMI.2023.3307688>.
- [21] A.A. Heydari, C.A. Thompson, A. Mehmood, Softadapt: techniques for adaptive loss weighting of neural networks with multi-part loss functions, *arXiv preprint arXiv:1912.12355*, 2019.
- [22] Q. Hong, J.W. Siegel, Q. Tan, J. Xu, On the activation function dependence of the spectral bias of neural networks, *arXiv preprint arXiv:2208.04924*, 2022.
- [23] X. Huang, T. Alkhalifah, Efficient physics-informed neural networks using hash encoding, *J. Comput. Phys.* 501 (2024) 112760.
- [24] A. Jacot, F. Gabriel, C. Hongler, Neural tangent kernel: convergence and generalization in neural networks, *Adv. Neural Inf. Process. Syst.* 31 (2018).
- [25] A.D. Jagtap, K. Kawaguchi, G.E. Karniadakis, Adaptive activation functions accelerate convergence in deep and physics-informed neural networks, *J. Comput. Phys.* 404 (2020) 109136.
- [26] X. Jin, S. Cai, H. Li, G.E. Karniadakis, Nsfnets (Navier-Stokes flow nets): physics-informed neural networks for the incompressible Navier-Stokes equations, *J. Comput. Phys.* 426 (2021) 109951.
- [27] A. Kammonen, L. Liang, A. Pandey, R. Tempone, Comparing spectral bias and robustness for two-layer neural networks: sgd vs adaptive random Fourier features, in: International Conference on Scientific Computing and Machine Learning, 2024.
- [28] G.E. Karniadakis, I.G. Kevrekidis, L. Lu, P. Perdikaris, S. Wang, L. Yang, Physics-informed machine learning, *Nat. Rev. Phys.* 3 (2021) 422–440.
- [29] S. Kaur, J. Cohen, Z.C. Lipton, On the maximum Hessian eigenvalue and generalization, in: Proceedings on, in: PMLR, 2023, pp. 51–65.
- [30] B.C. Koenig, S. Kim, S. Deng, KAN-ODEs: Kolmogorov-Arnold network ordinary differential equations for learning dynamical systems and hidden physics, *Comput. Methods Appl. Mech. Eng.* 432 (2024) 117397.
- [31] A.N. Kolmogorov, On the representation of continuous functions of many variables by superposition of continuous functions of one variable and addition, *Dokl. Akad. Nauk SSSR* 114 (1957) 953–956.
- [32] I.E. Lagaris, A. Likas, D.I. Fotiadis, Artificial neural networks for solving ordinary and partial differential equations, *IEEE Trans. Neural Netw.* 9 (1998) 987–1000.
- [33] J. Lee, L. Xiao, S. Schoenholz, Y. Bahri, R. Novak, J. Sohl-Dickstein, J. Pennington, Wide neural networks of any depth evolve as linear models under gradient descent, *Adv. Neural Inf. Process. Syst.* 32 (2019).
- [34] Z. Li, Kolmogorov-Arnold networks are radial basis function networks, *arXiv preprint arXiv:2405.06721*, 2024.
- [35] Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart, A. Anandkumar, Fourier neural operator for parametric partial differential equations, in: International Conference on Learning Representations, 2021.

- [36] S. Liang, L. Lyu, C. Wang, H. Yang, Reproducing activation function for deep learning, *Commun. Math. Sci.* (2024).
- [37] Z. Liao, M.W. Mahoney, Hessian eigenspectra of more realistic nonlinear models, *Adv. Neural Inf. Process. Syst.* 34 (2021) 20104–20117.
- [38] Y. Liu, R. Wu, Y. Jiang, Binary structured physics-informed neural networks for solving equations with rapidly changing solutions, 518, 113341, <https://linkinghub.elsevier.com/retrieve/pii/S0021999124005898>, <https://doi.org/10.1016/j.jcp.2024.113341>.
- [39] Z. Liu, Y. Wang, S. Vaidya, F. Ruehle, J. Halverson, M. Soljačić, T.Y. Hou, M. Tegmark, Kan: Kolmogorov-Arnold networks, *arXiv preprint arXiv:2404.19756*, 2024.
- [40] L. McClenny, U. Braga-Neto, Self-adaptive physics-informed neural networks using a soft attention mechanism, *arXiv preprint arXiv:2009.04544*, 2020.
- [41] L.D. McClenny, U.M. Braga-Neto, Self-adaptive physics-informed neural networks, *J. Comput. Phys.* 474 (2023) 111722.
- [42] M.A. Mendez, A. Ianiro, B.R. Noack, S.L. Brunton, *Data-Driven Fluid Mechanics: Combining First Principles and Machine Learning*, Cambridge University Press, 2023.
- [43] G. Montavon, K.R. Müller, *Neural networks-tricks of the trade*, second edition.
- [44] B. Moseley, A. Markham, T. Nissen-Meyer, Finite basis physics-informed neural networks (fbpinns): a scalable domain decomposition approach for solving differential equations, *Adv. Comput. Math.* 49 (2023) 62.
- [45] F. Mostajeran, R. Mokhtari, Deepbhp: deep neural network algorithm for solving backward heat conduction problems, *Comput. Phys. Commun.* 272 (2022) 108236, <https://doi.org/10.1016/j.cpc.2021.108236>.
- [46] N. Rahaman, A. Baratin, D. Arpit, F. Draxler, M. Lin, F. Hamprecht, Y. Bengio, A. Courville, On the spectral bias of neural networks, in: *International Conference on Machine Learning*, in: PMLR, 2019, pp. 5301–5310.
- [47] M. Raissi, P. Perdikaris, G.E. Karniadakis, Physics-informed neural networks: a deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations, *J. Comput. Phys.* 378 (2019) 686–707.
- [48] P. Ramachandran, B. Zoph, Q.V. Le, Searching for activation functions, *arXiv preprint arXiv:1710.05941*, 2017.
- [49] P. Rathore, W. Lei, Z. Frangella, L. Lu, M. Udell, Challenges in training pinns: a loss landscape perspective, *arXiv preprint arXiv:2402.01868*, 2024.
- [50] S. Rezaei, A. Harandi, A. Moineddin, B.X. Xu, S. Reese, A mixed formulation for physics-informed neural networks as a potential solver for engineering problems in heterogeneous domains: comparison with finite element method, *Comput. Methods Appl. Mech. Eng.* 401 (2022) 115616.
- [51] S. Rigas, M. Papachristou, T. Papadopoulos, F. Anagnostopoulos, G. Alexandridis, Adaptive training of grid-dependent physics-informed Kolmogorov-Arnold networks, *arXiv preprint arXiv:2407.17611*, 2024.
- [52] P. Roy, S.T. Castonguay, Exact enforcement of temporal continuity in sequential physics-informed neural networks, *Comput. Methods Appl. Mech. Eng.* 430 (2024) 117197.
- [53] S.T. Seydi, Unveiling the power of wavelets: a wavelet-based Kolmogorov-Arnold network for hyperspectral image classification, *arXiv preprint arXiv:2406.07869*, 2024.
- [54] K. Shukla, J.D. Toscano, Z. Wang, Z. Zou, G.E. Karniadakis, A comprehensive and fair comparison between mlp and kan representations for differential equations and operator networks, *J. Comput. Phys.* 42 (2024) 789–812.
- [55] Y. Song, H. Wang, H. Yang, M.L. Taccari, X. Chen, Loss-attentional physics-informed neural networks, *J. Comput. Phys.* 501 (2024) 112781.
- [56] S.S. Sidharth, A.R. Keerthana, R. Gokul, K.P. Anas, Chebyshev polynomial-based Kolmogorov-Arnold networks: an efficient architecture for nonlinear function approximation, *arXiv preprint arXiv:2405.07200*, 2024.
- [57] L.R. Süttfeld, F. Brieger, H. Finger, S. Füllhase, G. Pipa, Adaptive blending units: trainable activation functions for deep neural networks, in: *Intelligent Computing: Proceedings of the 2020 Computing Conference*, Volume 3, Springer, 2020, pp. 37–50.
- [58] H.T. Ta, Bsrbf-kan: a combination of b-splines and radial basic functions in Kolmogorov-Arnold networks, *arXiv preprint arXiv:2406.11173*, 2024.
- [59] M. Takamoto, T. Praditia, R. Leiteritz, D. MacKinlay, F. Alesiani, D. Pflüger, M. Niepert, Pdebench: an extensive benchmark for scientific machine learning, *Adv. Neural Inf. Process. Syst.* 35 (2022) 1596–1611.
- [60] M. Tancik, P. Srinivasan, B. Mildenhall, S. Fridovich-Keil, N. Raghavan, U. Singhal, R. Ramamoorthi, J. Barron, R. Ng, Fourier features let networks learn high frequency functions in low dimensional domains, *Adv. Neural Inf. Process. Syst.* 33 (2020) 7537–7547.
- [61] U. Trottenberg, C.W. Oosterlee, A. Schuller, *Multigrid*, Elsevier, 2000.
- [62] U.B. Waheed, Kronecker neural networks overcome spectral bias for pinn-based wavefield computation, *IEEE Geosci. Remote Sens. Lett.* 19 (2022) 1–5.
- [63] H. Wang, L. Lu, S. Song, G. Huang, Learning specialized activation functions for physics-informed neural networks, *arXiv preprint arXiv:2308.04073*, 2023.
- [64] J. Wang, X. Feng, H. Xu, Adaptive sampling points based multi-scale residual network for solving partial differential equations, *Comput. Math. Appl.* 169 (2024) 223–236.
- [65] S. Wang, Y. Teng, P. Perdikaris, Understanding and mitigating gradient flow pathologies in physics-informed neural networks, *SIAM J. Sci. Comput.* 43 (2021) A3055–A3081.
- [66] S. Wang, X. Yu, P. Perdikaris, When and why pinns fail to train: a neural tangent kernel perspective, *J. Comput. Phys.* 449 (2022) 110768.
- [67] Y. Wang, J.W. Siegel, Z. Liu, T.Y. Hou, On the expressiveness and spectral bias of kans, *arXiv preprint arXiv:2410.01803*, 2024.
- [68] Y. Wang, J. Sun, J. Bai, C. Anitescu, M.S. Eshaghi, X. Zhuang, T. Rabczuk, Y. Liu, Kolmogorov Arnold informed neural network: a physics-informed deep learning framework for solving pdes based on Kolmogorov Arnold networks, *arXiv preprint arXiv:2406.11045*, 2024.
- [69] Y. Wang, Y. Yao, J. Guo, Z. Gao, A practical pinn framework for multi-scale problems with multi-magnitude loss terms, *J. Comput. Phys.* 510 (2024) 113112.
- [70] Y. Wu, Z. Zhu, F. Liu, G. Chrysos, V. Cevher, Extrapolation and spectral bias of neural nets with Hadamard product: a polynomial net study, *Adv. Neural Inf. Process. Syst.* 35 (2022) 26980–26993.
- [71] J. Xu, Z. Chen, J. Li, S. Yang, W. Wang, X. Hu, E.C.H. Ngai, Fourierkan-gcf: Fourier Kolmogorov-Arnold network—an effective and efficient feature transformation for graph collaborative filtering, *arXiv preprint arXiv:2406.01034*, 2024.
- [72] Z.Q.J. Xu, Frequency principle: Fourier analysis sheds light on deep neural networks, *Commun. Comput. Phys.* 28 (2020) 1746–1767.
- [73] S. Yang, H. Kim, Y. Hong, K. Yee, R. Maulik, N. Kang, Data-driven physics-informed neural networks: a digital twin perspective, *Comput. Methods Appl. Mech. Eng.* 428 (2024) 117075.
- [74] Z. Yao, A. Gholami, K. Keutzer, M.W. Mahoney, Pyhessian: neural networks through the lens of the Hessian, in: *2020 IEEE International Conference on Big Data (Big Data)*, IEEE, 2020, pp. 581–590.
- [75] C. Zeng, J. Wang, H. Shen, Q. Wang, Kan versus mlp on irregular or noisy functions, *arXiv preprint arXiv:2408.07906*, 2024.
- [76] D. Zhang, Y. Li, S. Ying, Trans-net: a transferable pretrained neural networks based on temporal domain decomposition for solving partial differential equations, *Comput. Phys. Commun.* 299 (2024) 109130, <https://doi.org/10.1016/j.cpc.2024.109130>.
- [77] J. Zhang, C. Ding, Simple yet effective adaptive activation functions for physics-informed neural networks, *Comput. Phys. Commun.* (2024) 109428, <https://doi.org/10.1016/j.cpc.2024.109428>.
- [78] H. Zhao, Y. Hao, W. Xu, C. Li, Adaptive-sampling physics-informed neural network for viscoacoustic wavefield simulation, *IEEE Geosci. Remote Sens. Lett.* (2024).
- [79] Zhi-Qin John Xu, W.C. Lulu Zhang, On understanding and overcoming spectral biases of deep neural network learning methods for solving pdes, *J. Comput. Phys.* (2025).